ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΛΟΠΟΝΝΗΣΟΥ
ΣΧΟΛΗ ΟΙΚΟΝΟΜΙΑΣ ΚΑΙ ΤΕΧΝΟΛΟΓΙΑΣ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Πτυχιακή εργασία

Οπτικοποίηση Προαπαιτούμενων Οδηγού Σπουδών

Βάια Μαρία Γεγιάννη

Επιβλέπων:

Νίκος Πλατής
Επίκουρος Καθηγητής

Τρίπολη, Οκτώβριος 2026

Visualization of Course Prerequisites in an Academic Curriculum

Περίληψη

Στην παρούσα εργασία αναπτύξαμε μια διαδικτυακή εφαρμογή για την οπτικοποίηση των μαθημάτων και των σχέσεων προαπαιτούμενων του οδηγού σπουδών του Τμήματος Πληροφορικής και Τηλεπικοινωνίων του Πανεπιστήμιου Πελοποννήσου.

Η εφαρμογή παρουσιάζει τα μαθήματα ως κόμβους ενός γράφου και τις σχέσεις προαπαιτούμενων ως συνδέσεις μεταξύ των κόμβων. Παρέχει τη δυνατότητα αναζήτησης και φιλτραρίσματος, την ανάλυση των προαπαιτούμενων ενός επιλεγμένου μαθήματος, και τον προγραμματισμό των σπουδών με βάση τα μαθήματα που έχει ολοκληρώσει επιτυχώς ο φοιτητής.

Σκοπός της εφαρμογής είναι η ευκολότερη κατανόηση του προγράμματος σπουδών, η εξερεύνηση των μαθημάτων και των συνδέσεών τους καθώς και η υποστήριξη των φοιτητών στην οργάνωση της ακαδημαϊκής τους πορείας.

Λέξεις Κλειδιά: Οπτικοποίηση, Γράφος, Διαδραστική Εφαρμογή, Οδηγός Σπουδών, Προαπαιτούμενα, PHP, SQLite, HTML, CSS, JavaScript, D3.js

Abstract

The purpose of this work is to develop an online application for visualizing the courses and their prerequisite relationships within the curriculum of the Department of Informatics and Telecommunications of the University of the Peloponnese.

The application displays the courses as nodes in a graph and the prerequisite relationships as connections between those nodes. The application provides the ability to search and filter, to analyze the prerequisites for a selected course, and supports academic planning based on the courses the student has already successfully completed.

The purpose of this application is to facilitate the better understanding of the curriculum structure, to help students explore the relationships between courses and to support them in planning their academic journey.

Keywords: Visualization, Graph, Interactive Application, Curriculum, Prerequisites, PHP, SQLite, HTML, CSS, JavaScript, D3.js

Περιεχόμενα

Περίληψη **1**

Abstract **2**

Ευχαριστίες **6**

Κεφάλαιο 1 7

Εισαγωγή **7**

1.1 Περιγραφή του προβλήματος και επισκόπηση της λύσης 7

1.2 Επισκόπηση κειμένου 8

Κεφάλαιο 2 10

Απαιτήσεις συστήματος **10**

2.1 Χρήστες συστήματος 10

2.2 Καταγραφή απαιτήσεων 10

2.2.1 Λειτουργικές απαιτήσεις 10

2.2.2 Μη-Λειτουργικές απαιτήσεις 11

2.3 Σενάρια χρήσης 11

2.4 Παράγοντες σχεδιασμού 13

Κεφάλαιο 3 14

Επίδειξη της εφαρμογής **14**

3.1 Κεντρικό περιβάλλον της εφαρμογής 14

3.2 Πίνακας ελέγχου και φίλτρα 15

3.2.1 Λειτουργίες φιλτραρίσματος 16

3.3 Focus Mode 19

3.4 Planning Mode και λειτουργία Drag Selection 19

3.5 Panel πληροφοριών μαθήματος 21

3.6 Tooltips 23

Κεφάλαιο 4 24

Βάση δεδομένων και τεχνολογίες υλοποίησης **24**

4.1 Βάση δεδομένων 24

4.2 Γλώσσες υλοποίησης 26

4.2.1 PHP 26

4.2.2 HTML 26
4.2.3 CSS 27
4.2.4 JavaScript 27
4.3 Βιβλιοθήκες και εργαλεία 28
4.3.1 D3.js 28
4.3.2 PDO 28
4.3.3 SQLite 29
4.3.4 JSON 29
Κεφάλαιο 5 30
Αρχιτεκτονική συστήματος και υλοποίηση 30
5.1 Αρχιτεκτονική συστήματος 30
5.2 Σύνδεση με τη βάση και ανάκτηση δεδομένων 31
5.3 Φόρτωση δεδομένων γράφου 33
5.4 Δημιουργία γράφου 33
5.5 Προετοιμασία κόμβων και συνδέσεων 34
5.6 Προσομοίωση γράφου (Graph Simulation) 36
5.6.1 Force Simulation 36
5.6.2 Περιορισμός και κατανομή κόμβων ανά ακαδημαϊκό έτος 37
5.6.3 Ενημέρωση θέσεων κόμβων (Tick Function) 39
5.7 Μηχανισμοί φιλτραρίσματος 40
5.7.1 Ακαδημαϊκό έτος 40
5.7.2 Κατηγορία μαθήματος 40
5.7.3 Διαθεσιμότητα μαθήματος 40
5.7.4 Συνδυασμός συναρτήσεων φιλτραρίσματος 41
5.8 Μηχανισμός αναζήτησης 41
5.9 Focus Mode 41
5.10 Planning Mode 45
5.10.1 Μηχανισμός Drag Selection 49
5.11 Ενημέρωση οπτικής κατάστασης 52
5.12 Βοηθητικά υπομνήματα και επεξηγηματικά tooltips 53
5.12.1 Legends 53
5.12.2 Tooltips 54

Κεφάλαιο 6 56
Εξέλιξη υλοποίησης και αρχικές ιδέες σχεδιασμού 56
6.1 Αρχική δομή του γράφου 56
6.2 Εναλλακτικές προσεγγίσεις οπτικοποίησης 57
6.3 Λειτουργία μετακίνησης κόμβων 58
6.4 Ετικέτες κόμβων 58
6.5 Χρωματικές παλέτες εφαρμογής 59
Κεφάλαιο 7 61
Συμπεράσματα και μελλοντικές βελτιώσεις 61
7.1 Συμπεράσματα 61
7.2 Μελλοντικές βελτιώσεις 61
Παράρτημα Α 63
Τοπική εκτέλεση της εφαρμογής 63
Βιβλιογραφία 64

Ευχαριστίες

Ολοκληρώνοντας την παρούσα πτυχιακή εργασία, θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή μου, τον κ. Πλατή, για την καθοδήγηση και την βοήθεια που μου παρείχε, καθώς και για την εμπιστοσύνη που μου έδειξε κατά την διάρκεια υλοποίησης της πτυχιακής εργασίας.

Επίσης, ευχαριστώ θερμά την οικογένεια και τους φίλους μου για την στήριξη και ενθάρρυνση που μου δώσανε σε όλη την διάρκεια των σπουδών μου.

Κεφάλαιο 1

Εισαγωγή

1.1 Περιγραφή του προβλήματος και επισκόπηση της λύσης

Τα σύγχρονα προγράμματα σπουδών των ανώτατων εκπαιδευτικών ιδρυμάτων, συχνά περιέχουν πληθώρα μαθημάτων, τα περισσότερα από τα οποία συνδέονται μέσω των προαπαιτούμενων σχέσεων τους. Κατ΄επέκταση, οι οδηγοί σπουδών γίνονται πιο περίπλοκοι, με αποτέλεσμα οι φοιτητές συχνά να αντιμετωπίζουν δυσκολίες στην κατανόηση των σχέσεων μεταξύ των μαθήματων, στην αναγνώριση της σωστής σειράς επιλογής μαθήματων και στην οργάνωση της ακαδημαϊκής τους πορείας με αποτελεσματικό τρόπο. Στους περισσοτέρους οδηγούς σπουδών, η πληροφορία των προαπαιτούμενων αναπαρίσταται σε μορφή πινάκων ή με περιγραφικό κείμενο, δυσκολεύοντας τους φοιτητές να καταλάβουν γρήγορα και εύκολα την δομή του προγράμματος σπουδών.

Στην παρούσα εργασία αναπτύξαμε και σχεδιάσαμε ένα διαδικτυακό σύστημα για την οπτικοποίηση των μαθήματων του οδηγού σπουδών και την οργάνωση της ακαδημαϊκής πορείας των φοιτητών. Κύριος στόχος της εφαρμογής είναι να βοηθήσει τους φοιτητές να κατανοήσουν καλυτέρα πώς τα προαπαιτούμενα μαθήματα συνδέονται μεταξύ τους, καθώς και να τους διευκολύνει στην οργάνωση της πορείας των σπουδών τους. Το σύστημα στοχεύει στο να παρέχει στους φοιτητές έναν πιο εύχρηστο τρόπο να εξερευνήσουν τη δομή του προγράμματος σπουδών μέσω ενός διαδραστικού γραφήματος.

Στο διαδραστικό γράφημα, κάθε μάθημα απεικονίζεται στην μορφή ενός κόμβου και οι σχέσεις των προαπαιτούμενων μαθήματων απεικονίζονται ως ενώσεις μεταξύ των κόμβων. Η εφαρμογή υποστηρίζει δύο κύριες λειτουργίες αλληλεπιδράσεις με το γράφημα: την λειτουργία ανάλυσης προαπαιτούμενων, focus mode, και την λειτουργία οργάνωσης μαθημάτων, planning mode.

Μέσω του focus mode, οι φοιτητές μπορούν να επιλέξουν ένα μάθημα και το γράφημα θα φανερώσει τα προαπαιτούμενα του συγκεκριμένου μαθήματος, καθώς και τα μαθήματα που μπορούν μελλοντικά να επιλέξουν, με την προϋπόθεση ότι έχουν το συγκεκριμένο μάθημα ως προαπαιτούμενο. Η συγκεκριμένη λειτουργία έχει ως στόχο να βοηθήσει τους φοιτητές να κατανοήσουν καλύτερα τις σχέσεις των μαθήματων του οδηγού σπουδών.

Μέσω του planning mode, οι φοιτητές μπορούν να επιλέξουν τα μαθήματα που έχουν ήδη ολοκληρώσει επιτυχώς. Με βάση αυτών των επιλογών, η εφαρμογή δυναμικά αναγνωρίζει ποια μαθήματα μπορεί να επιλέξει ο χρήστης να εγγραφτεί και ποια μαθήματα δεν μπορεί

ακόμα να επιλέξει λόγω των προαπαιτούμενων μαθήματων τους. Η συγκεκριμένη λειτουργία στοχεύει στην απλοποίηση της διαδικασίας του ακαδημαϊκού προγραμματισμού.

Κατ΄επέκταση, το σύστημα παρέχει μηχανισμούς φιλτραρίσματος και αναζήτησης για την εύκολη πλοήγηση του χρήστη. Οι μηχανισμοί αυτοί, επιτρέπουν στον χρήστη να εστιάζει σε συγκεκριμένα ακαδημαϊκά έτη και κατηγορίες μαθήματων. Οι φοιτητές μπορούν δυναμικά να φιλτράρουν τα μαθήματα με βάση το έτος, την κατηγορία και την διαθεσιμότητα, ενώ η λειτουργία αναζήτησης επιτρέπει τον γρήγορο εντοπισμό των μαθήματων μέσα στο γράφημα.

Για την υλοποίηση της εφαρμογής χρησιμοποιήθηκαν κάποιες γλώσσες - βιβλιοθήκες HTML, CSS, Javascript, PHP, και την βιβλιοθήκη οπτικοποίησης D3.js. Το σύστημα χρησιμοποιεί SQLlite μέσω PHP Data Objects (PDO) για την συλλογή δεδομένων και πληροφοριών από την βάση δεδομένων. Παράλληλα, JSON χρησιμοποιήθηκε για την ανταλλαγή των δεδομένων μεταξύ του backend συστήματος και του γραφικού περιβάλλοντος οπτικοποίησης. Η βάση δεδομένων και οι πληροφορίες των μαθημάτων που χρησιμοποιήθηκαν στην εφαρμογή, παρασχέθηκαν από το τμήμα της Πληροφορικής και Τηλεπικοινωνίων του Πανεπιστήμιου της Πελοποννήσου, και αντικατοπτρίζουν με ακρίβεια τον επίσημο οδηγό σπουδών του τμήματος.

Ιδιαίτερη έμφαση δόθηκε στην ευχρηστία, στην σαφή οπτική αναπαράσταση των σχέσεων των προαπαιτούμενων μαθήματων και στην διαδραστικότητα του συστήματος.

1.2 Επισκόπηση κειμένου

Η αναφορά της εργασίας έχει οργανωθεί σε εφτά βασικά κεφάλαια, καθένα από τα οποία παρουσιάζει διαφορετικές πτυχές της ανάπτυξης, της λειτουργικότητας και του σχεδιασμού της εφαρμογής.

Το Κεφάλαιο 1 είναι μια εισαγωγή στο θέμα της εργασίας, στους στόχους της εφαρμογής και της σημαντικότητας της οπτικοποίησης των προαπαιτούμενων του οδηγού σπουδών με σκοπό να υποστηρίξει τους φοιτητές κατά την διάρκεια των σπουδών τους στο τμήμα της Πληροφορικής και Τηλεπικοινωνίων.

Το Κεφάλαιο 2 αναλύει τις λειτουργικές και μη-λειτουργικές απαιτήσεις του συστήματος και περιγράφει τις λειτουργίες της εφαρμογής. Ακόμα, περιλαμβάνει τα προβλήματα που το σύστημα επιχειρεί να λύσει καθώς και ποιες ανάγκες των χρηστών ληφθήκαν υπόψιν κατά τον σχεδιασμό της εφαρμογής.

Το Κεφάλαιο 3 παρουσιάζει το τελικό περιβάλλον της εφαρμογής και τις βασικές λειτουργίες του συστήματος μέσω στιγμιότυπων οθόνης και παραδειγμάτων χρήσης.

Το Κεφάλαιο 4 περιγράφει τη βάση δεδομένων που χρησιμοποιείται για την απεικόνιση των δεδομένων. Περιγράφονται όλοι οι πίνακες που χρησιμοποιήθηκαν κατά την υλοποίηση της εφαρμογής καθώς και τα χαρακτηριστικά τους. Ακόμα, αναλύει τις τεχνολογίες και τα εργαλεία που χρησιμοποιήθηκαν στην ανάπτυξη της εφαρμογής.

Το Κεφάλαιο 5 επεκτείνεται στην αρχιτεκτονική του συστήματος. Περιγράφει αναλυτικά την οργάνωση της εφαρμογής, την δομή του γραφήματος οπτικοποίησης και, γενικότερα, τον τρόπο λειτουργίας των μηχανισμών του συστήματος.

Το Κεφάλαιο 6 περιγράφει την εξέλιξη της εφαρμογής κατά την διάρκεια την υλοποίησης της. Παρουσιάζει τις αρχικές ιδέες διεπαφής και κάποιες προσεγγίσεις οπτικοποίησης του γραφήματος που αργότερα απορρίφθηκαν για το τελικό μοντέλο της εφαρμογής.

Το Κεφάλαιο 7 παρουσιάζει τα συμπεράσματα της εργασίας και πιθανές μελλοντικές βελτιώσεις που θα μπορούσαν να γίνουν στην εφαρμογή.

Τέλος, στο παράρτημα Α δίνονται οι οδηγίες εγκατάστασης και τοπικής εκτέλεσης της εφαρμογής.

Κεφάλαιο 2

Απαιτήσεις συστήματος

Οι απαιτήσεις του συστήματος βοηθούν να καθορίσουμε και να κατανοήσουμε καλυτέρα το πρόβλημα που η εφαρμογή καλείται να επιλύσει. Οι απαιτήσεις μπορούν ακόμα να θεωρηθούν ως περίγραφες των λειτουργιών και δυνατοτήτων τις οποίες το σύστημα πρέπει να παρέχει για να εξυπηρετήσει τον σκοπό για τον οποίο αναπτύχθηκε. Για αυτούς τους λόγους, η καταγραφή αυτών των απαιτήσεων συμβάλλει στην σωστή σχεδίαση και υλοποίηση του συστήματος ενώ παράλληλα μειώνει τις πιθανότητες ανάπτυξης λαθών.

2.1 Χρήστες συστήματος

Η εφαρμογή αναπτύχθηκε, κατά κύριο λόγο, για τους φοιτητές του τμήματος Πληροφορικής και Τηλεπικοινωνίων του Πανεπιστήμιου της Πελοποννήσου. Οι χρήστες αλληλοεπιδρούν με την εφαρμογή μέσω ενός γραφήματος με σκοπό να κατανοήσουν τις σχέσεις μεταξύ των προαπαιτούμενων μαθήματων και να οργανώσουν το ακαδημαϊκό πρόγραμμά τους.

Αφού η εφαρμογή επικεντρώνεται στην οπτικοποίηση των προαπαιτούμενων του οδηγού σπουδών, το σύστημα δεν περιέχει πολλαπλούς ρόλους χρηστών, ούτε μηχανισμούς επαλήθευσης ταυτότητας χρήστη. Όλες οι λειτουργίες είναι άμεσα προσβάσιμες μέσω της διεπαφής του συστήματος.

2.2 Καταγραφή απαιτήσεων

2.2.1 Λειτουργικές απαιτήσεις

Οι λειτουργικές απαιτήσεις του συστήματος περιγράφονται στον παρακάτω πίνακα.

Πίνακας 1: Λειτουργικές απαιτήσεις συστήματος

Λειτουργικές απαιτήσεις	Προτεραιότητα
Ο χρήστης θα μπορεί να δει όλα τα μαθήματα του οδηγού σπουδών σε μορφή γράφου.	Υψηλή
Ο χρήστης θα μπορεί να δει τα προαπαιτούμενα κάθε μαθήματος σε μορφή γράφου.	Υψηλή

Ο χρήστης επιλέγοντας ένα μάθημα θα μπορεί να δει τα προαπαιτούμενα του, καθώς και ποια άλλα μαθήματα έχουν αυτό ως προαπαιτούμενο.	Υψηλή
Ο χρήστης θα μπορεί να κάνει αναζήτηση κάποιο μάθημα με το όνομα ή κωδικό του μαθήματος.	Υψηλή
Ο χρήστης θα μπορεί να φιλτράρει τα μαθήματα ανά έτος, κατηγορία και διαθεσιμότητα.	Υψηλή
Ο χρήστης θα μπορεί να δει επιπρόσθετες λεπτομέρειες για ένα μάθημα επιλέγοντάς το.	Υψηλή
Ο χρήστης θα μπορεί να σημειώνει ποια μαθήματα έχει ολοκληρώσει επιτυχώς.	Υψηλή
Ο χρήστης θα μπορεί να σύρει τον κέρσορα για την πολλαπλή επιλογή μαθημάτων (drag selection)	Μέτρια
Η εφαρμογή θα δείχνει ποια μαθήματα μπορεί να πάρει ο χρήστης με βάση τα μαθήματα που έχει ολοκληρώσει επιτυχώς.	Υψηλή
Ο χρήστης θα μπορεί να αλλάξει την γλώσσα της εφαρμογής (Ελληνικά ή Αγγλικά)	Χαμηλή

2.2.2 Μη-Λειτουργικές απαιτήσεις

Οι μη-λειτουργικές απαιτήσεις του συστήματος περιγράφονται στον παρακάτω πίνακα.

Πίνακας 2: Μη-Λειτουργικές απαιτήσεις συστήματος

Μη-Λειτουργικές απαιτήσεις	**Προτεραιότητα**
Η εφαρμογή θα πρέπει να λειτουργεί σε όλα τα σύγχρονα browser (Chrome, Firefox, Safari, etc.)	Υψηλή
Η εφαρμογή θα πρέπει να διασφαλίζει την ακεραιότητα των δεδομένων.	Υψηλή
Η εφαρμογή θα πρέπει να ανανεώνεται δυναμικά όταν αλλάζουν τα δεδομένα.	Υψηλή
Η εφαρμογή θα πρέπει να είναι εύχρηστη και οπτικά κατατοπιστική για τον χρήστη.	Υψηλή

2.3 Σενάρια χρήσης

Τα σενάρια χρήσης περιγράφουν μια σειρά ενεργειών και γεγονότων που συμβαίνουν κατά την χρήση του συστήματος. Πρόκειται για περιγραφή σε μορφή κειμένου που αναφέρεται σε μία τρέχουσα ή σχεδιαζόμενη χρήση. Παρακάτω περιγράφονται ενδεικτικά κάποια σενάρια χρήσης του συστήματός μας.

Πίνακας 3: Σενάριο 1

Όνομα σεναρίου:	Εξερεύνηση προαπαιτούμενων μαθημάτων
Ροή γεγονότων:	1. Ο χρήστης ανοίγει την εφαρμογή. 2. Το γράφημα οπτικοποίησης των προαπαιτούμενων του οδηγού σπουδών εμφανίζεται. 3. Ο χρήστης επιλέγει έναν κόμβο που αντιστοιχεί σε ένα μάθημα. 4. Το σύστημα μαρκάρει το μάθημα αλλάζοντας το χρώμα του κόμβου. Εμφανίζει τα προαπαιτούμενα και τα μαθήματα που ξεκλειδώνονται με την επιτυχή ολοκλήρωση του επιλεγμένου μαθήματος. 5. Ο χρήστης διακρίνει με ευκολία τις σχέσεις των προαπαιτούμενων μέσω του οπτικού γράφου.

Πίνακας 4: Σενάριο 2

Όνομα σεναρίου:	Οργάνωση ακαδημαϊκής πορείας
Ροή γεγονότων:	1. Ο χρήστης ενεργοποιεί το planning mode. 2. Ο χρήστης επιλέγει στο γράφημα τα μάθημα που έχει ήδη ολοκληρώσει επιτυχώς. 3. Το σύστημα δυναμικά ξεχωρίζει με διαφορετικά χρώματα ποια μαθηματα μπορεί ο χρήστης να επιλέξει και ποια ακόμα δεν μπορεί. 4. Ο χρήστης χρησιμοποιεί την οπτικοποίηση για να οργανώσει ποια μαθήματα θα επιλέξει να παρακολουθήσει μελλοντικά.

Πίνακας 5: Σενάριο 3

Όνομα σεναρίου:	Αναζήτηση μαθήματων
Ροή γεγονότων:	1. Ο χρήστης εισάγει το όνομα ή τον κωδικό ενός μαθήματος στο πλαίσιο αναζήτησης. 2. Το σύστημα αναζητά και επιλέγει το μάθημα που ψάχνει ο χρήστης αλλάζοντας το χρώμα του συγκεκριμένου κόμβου. 3. Το σύστημα εμφανίζει αυτόματα τα προαπαιτούμενα του επιλεγμένου μαθήματος και ποια μαθήματα αυτό ξεκλειδώνει.

Πίνακας 6: Σενάριο 4

Όνομα σεναρίου:	Φιλτράρισμα μαθήματων
Ροή γεγονότων:	1. Ο χρήστης επιλέγει ποια φίλτρα θέλει να χρησιμοποιηθούν στο γράφημα. 2. Με βάση τις επιλογές του χρήστη, το γράφημα άμεσα αλλάζει οπτικά, είτε φανερώνοντας είτε κρύβοντας κόμβους. 3. Ο χρήστης μπορεί να αλληλεπιδράσει με ένα εμφανισιακά πιο απλό γράφημα.

Πίνακας 7: Σενάριο 5

Όνομα σεναρίου:	Εμφάνιση πληροφοριών μαθημάτων
Ροή γεγονότων:	1. Ο χρήστης κάνει διπλό κλικ στον κόμβο ενός μαθήματος. 2. Το σύστημα εμφανίζει ένα παράθυρο με σχετικές πληροφορίες για το συγκεκριμένο μάθημα. 3. Ο χρήστης γρήγορα παίρνει τις πληροφορίες που θέλει.

2.4 Παράγοντες σχεδιασμού

Τα σενάρια χρήσης περιγράφουν μια σειρά ενεργειών και γεγονότων που συμβαίνουν κατά την χρήση του συστήματος

Όσον αφορά το σχέδιο και την εμφάνιση της εφαρμογής, ιδιαίτερη έμφαση δόθηκε στην ευχρηστία και στην αλληλεπίδραση του γραφήματος. Λαμβάνοντας υπόψιν πως το σύστημα είναι βασισμένο στην οπτική παρουσίαση των προαπαιτούμενων σχέσεων μεταξύ των μαθήματων, η οπτική εμφάνιση του γραφήματος ήταν σημαντικό να παραμείνει καθαρή και κατανοητή, ακόμα και όταν απεικονίζεται μεγάλος αριθμός μαθήματων και συνδέσεων.

Για αυτόν τον λόγο, διάφορες αλληλεπιδράσεις και σχεδιασμοί διεπαφής ληφθήκαν υπόψιν, όπως ο μηχανισμός του δυναμικού φιλτραρίσματος (Ενότητα 5.7), η εμφάνιση κατατοπιστικών εξηγήσεων της εφαρμογής μέσω tooltips (Ενότητα 5.12), και χρωματικές αλλαγές στους κόμβους για εύκολη διάκριση (Ενότητα 6.5).

Κεφάλαιο 3

Επίδειξη της εφαρμογής

Σε αυτό το κεφάλαιο παρουσιάζεται το τελικό γραφικό περιβάλλον της εφαρμογής. Τα στιγμιότυπα οθόνης που περιλαμβάνονται απεικονίζουν την οπτικοποίηση του γραφήματος και τα βασικά στοιχεία του περιβάλλοντος χρήστη.

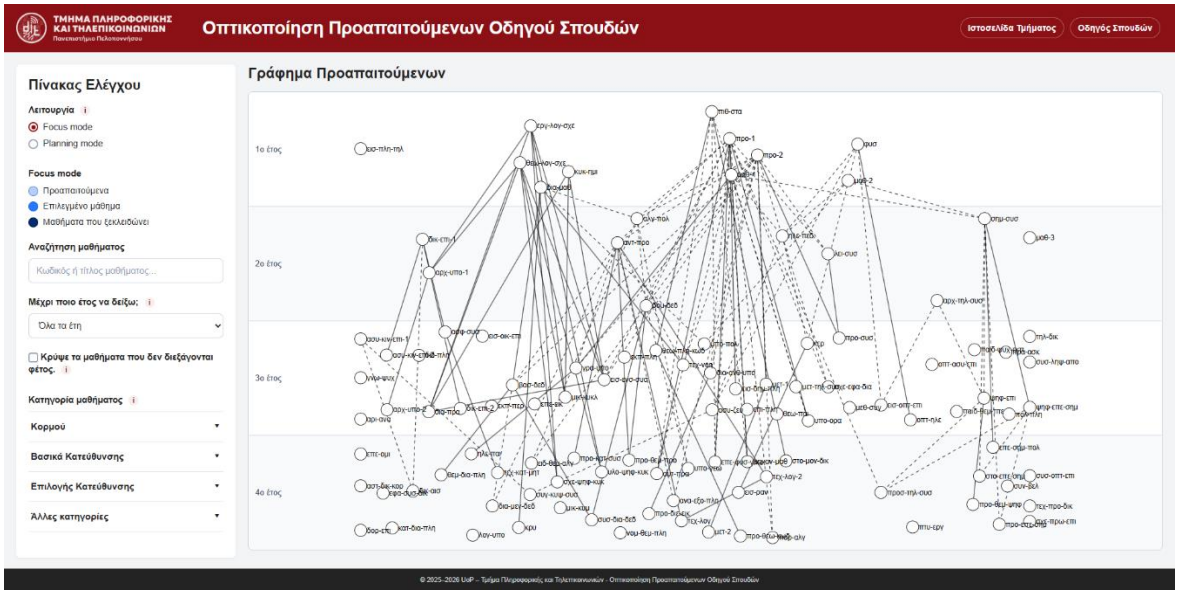
3.1 Κεντρικό περιβάλλον της εφαρμογής

Το βασικό περιβάλλον της εφαρμογής αποτελείται από δύο κύρια τμήματα:

- την περιοχή οπτικοποίησης του γραφήματος
- τον Πίνακα Ελέγχου

Στην περιοχή του γράφου εμφανίζονται οι σχέσεις προαπαιτούμενων μεταξύ των μαθημάτων. Οι συνεχόμενες γραμμές δηλώνουν σχέσεις προαπαιτούμενων τύπου ΚΑΙ (AND), ενώ οι διακεκομμένες γραμμές δηλώνουν σχέσεις τύπου Ή (OR) μεταξύ των μαθημάτων.

Στον Πίνακα Ελέγχου βρίσκονται οι επιλογές φιλτραρίσματος και μηχανισμού αλληλεπίδρασης.



Κεντρικό περιβάλλον της εφαρμογής

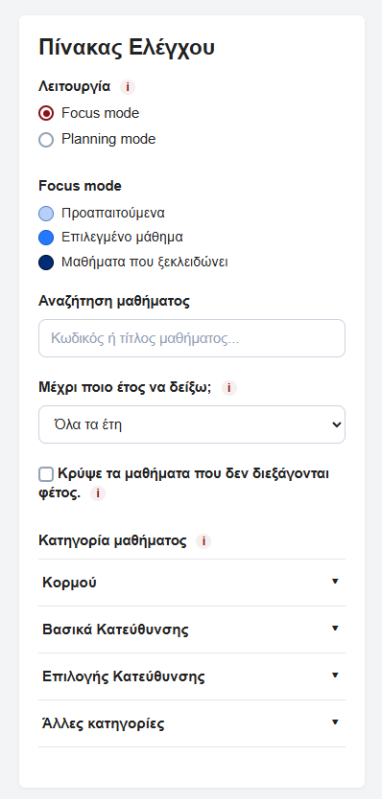
3.2 Πίνακας ελέγχου και φίλτρα

Ο Πίνακας Ελέγχου περιέχει τους μηχανισμούς φιλτραρίσματος και τα εργαλεία αλληλεπίδρασης της εφαρμογής.

Πιο συγκεκριμένα οι χρήστες μπορούν:

- να αλλάζουν την λειτουργία αλληλεπίδρασης με το γράφημα
- να αναζητούν συγκεκριμένα μαθήματα
- να φιλτράρουν τα μαθήματα ανά ακαδημαϊκό έτος
- να εμφανίζουν μόνο τα προσφερόμενα μαθήματα
- να φιλτράρουν τα μαθήματα ανά κατηγορία

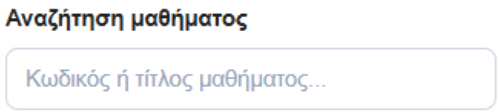
Ο Πίνακας Ελέγχου ενημερώνει δυναμικά την οπτικοποίηση του γράφου σύμφωνα με τα επιλεγμένα φίλτρα.



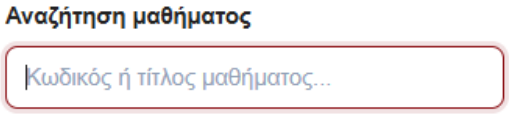
Πίνακας Ελέγχου

3.2.1 Λειτουργίες φιλτραρίσματος

Στα παρακάτω στιγμιότυπα απεικονίζεται το πεδίο της αναζήτησης:

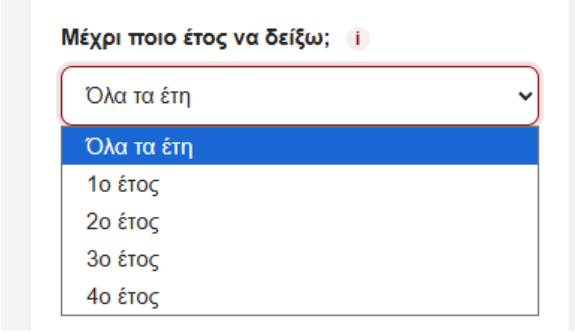


Πεδίο αναζήτησης μαθημάτων



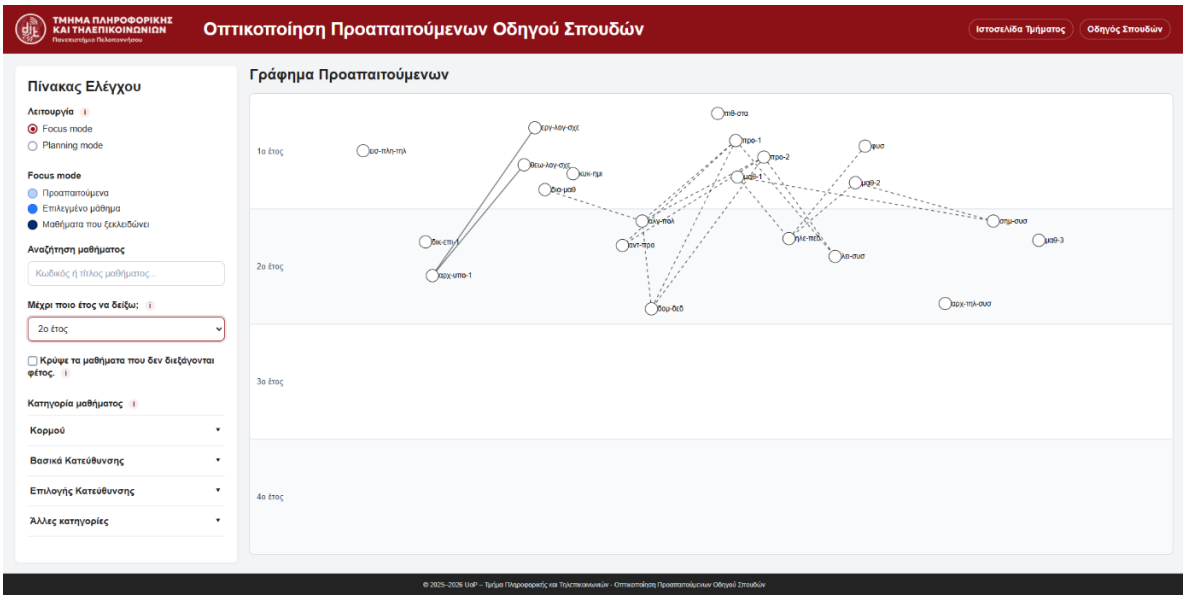
Εισαγωγή κειμένου στο πεδίο αναζήτησης

Ακολουθεί στιγμιότυπο με την dropdown επιλογή του ακαδημαϊκού έτους που επιθυμεί ο χρήστης να εμφανίσει το γράφημα. Το φίλτρο λειτουργεί προοδευτικά και εμφανίζει τα μαθήματα μέχρι και το εκτός που επιλέγει ο χρήστης:



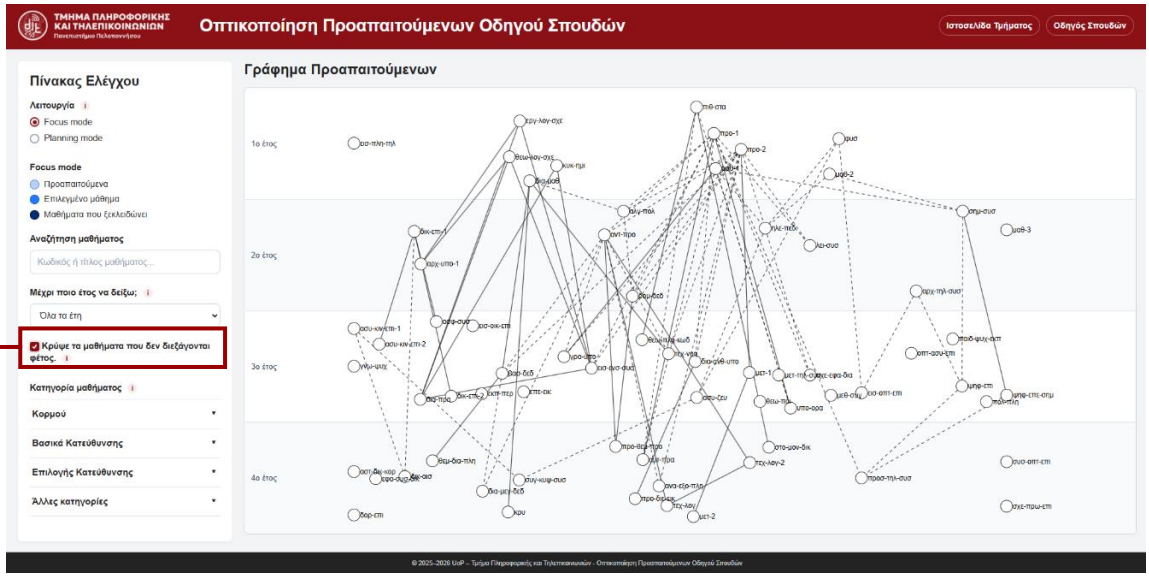
Dropdown επιλογής ακαδημαϊκού έτους

Παρακάτω φαίνεται ένα παράδειγμα οπτικοποίησης του γραφήματος με την επιλογή να εμφανιστεί μέχρι και το 2^{ο} ακαδημαϊκό έτος:

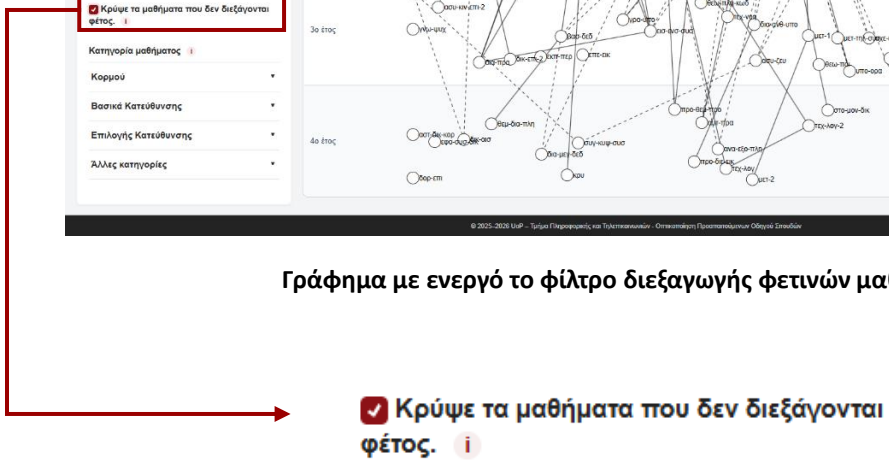


Γράφημα με ενεργό το φίλτρο ακαδημαϊκού έτους

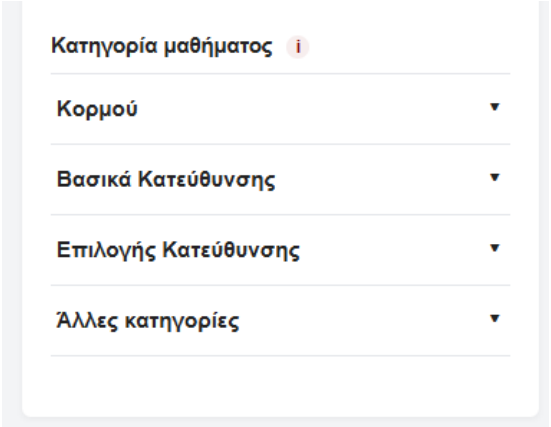
Παράδειγμα οπτικοποίησης του γραφήματος με την επιλογή να εμφανιστούν μόνο τα μάθημα που διεξάγονται το τρέχον ακαδημαϊκό έτος:



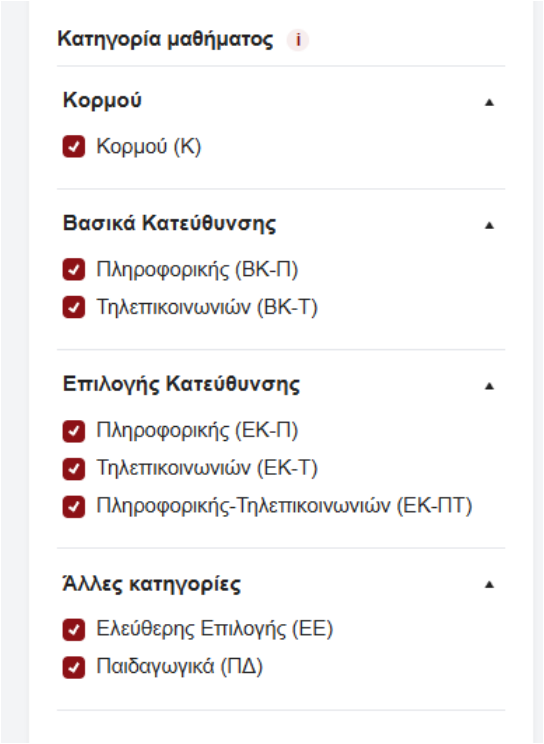
Γράφημα με ενεργό το φίλτρο διεξαγωγής φετινών μαθημάτων



Στα παρακάτω στιγμιότυπα απεικονίζεται το accordion με τα φίλτρα κατηγοριών των μαθημάτων:



Κλειστό accordion φίλτρου κατηγορίας μαθήματος



Ανοιχτό accordion φίλτρου κατηγορίας μαθήματος

3.3 Focus Mode

Το Focus Mode δίνει την δυνατότητα στους χρήστες να εξερευνούν την δομή προαπαιτούμενων ενός επιλεγμένου μαθήματος. Όταν επιλέγεται ένα μάθημα, η εφαρμογή επισημαίνει:

- το επιλεγμένο μάθημα
- τα προαπαιτούμενα μαθήματα
- και τα μαθήματα που ξεκλειδώνονται μέσω αυτού

Οι μη σχετικοί κόμβοι και συνδέσεις εμφανίζονται με μειωμένη φωτεινότητα, ώστε να βελτιώνεται η αναγνωσιμότητα της δομής που έχει επιλέξει ο χρήστης να εμφανιστεί, χωρίς να του αποσπάται η προσοχή.



Γράφημα με την χρήση του Focus Mode

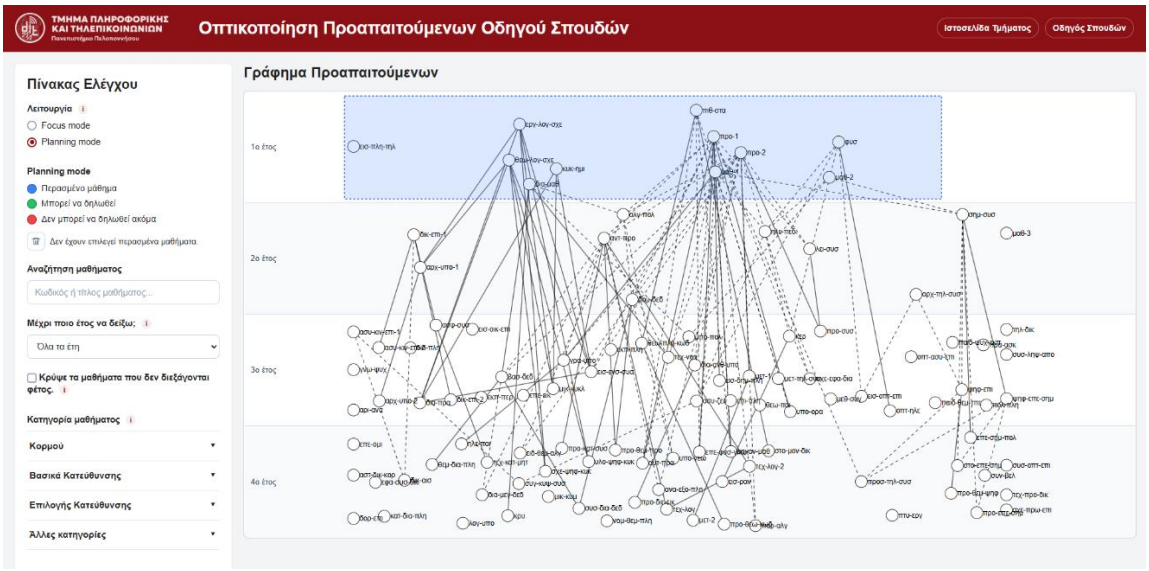
3.4 Planning Mode και λειτουργία Drag Selection

Το Planning Mode επιτρέπει στους φοιτητές να επιλέγουν τα μαθήματα που έχουν ήδη ολοκληρώσει και με βάση αυτά, η εφαρμογή υπολογίζει δυναμικά ποια μαθήματα είναι διαθέσιμα σύμφωνα με τους κανόνες προαπαιτούμενων του προγράμματος σπουδών.

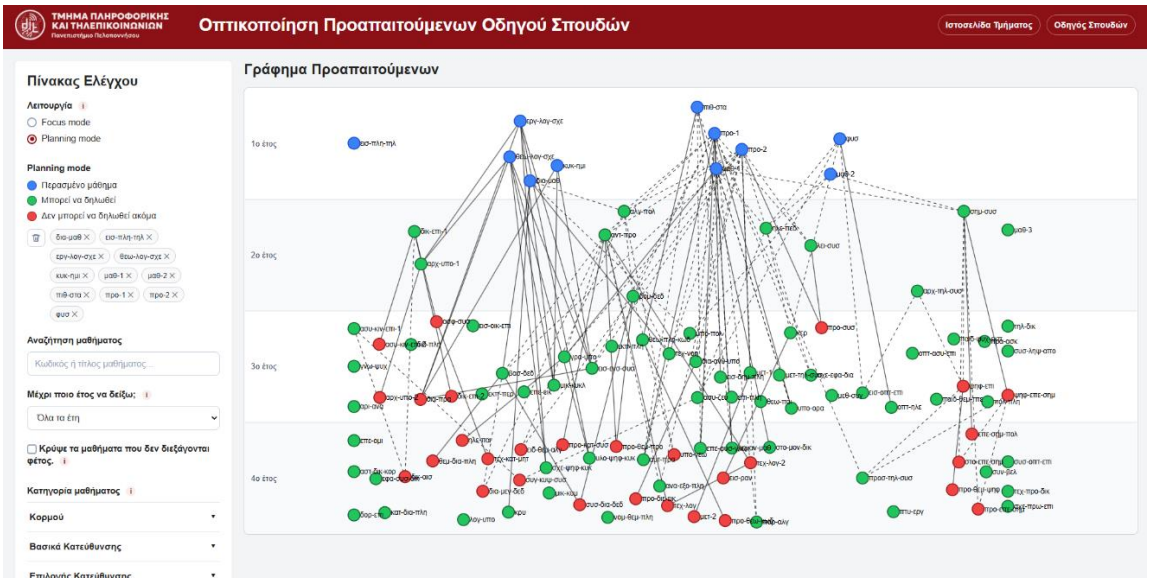
Πιο συγκεκριμένα, διαφορετικά χρώματα χρησιμοποιούνται για τον διαχωρισμό:

- των ολοκληρωμένων μαθημάτων
- των διαθέσιμων μαθημάτων
- και των μη διαθέσιμων μαθημάτων

Όσο η εφαρμογή βρίσκεται στο Planning Mode, υποστηρίζεται επίσης η λειτουργία drag selection. Οι χρήστες μπορούν να επιλέγουν πολλαπλά μαθήματα ταυτόχρονα, σχεδιάζοντας ένα ορθογώνιο επιλογής πάνω στον γράφο.

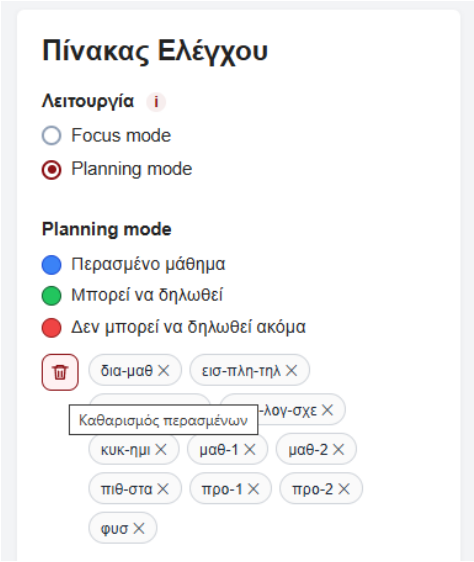


Χρήση drag selection κατά τη διαδικασία επιλογής μαθημάτων



Μαθήματα που επιλέχθηκαν μέσω drag selection

Τα ολοκληρωμένα μαθήματα εμφανίζονται στον Πίνακα Ελέγχου σε μορφή tags, επιτρέποντας στον χρήστη να παρακολουθεί εύκολα τις τρέχουσες επιλογές του.



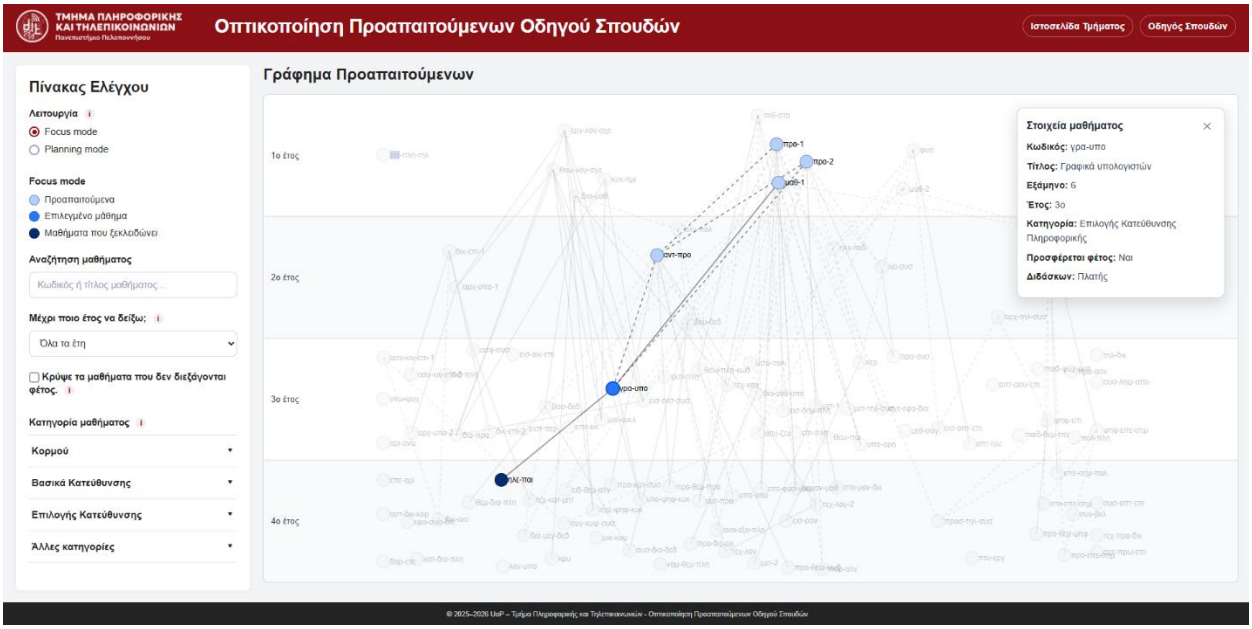
Εμφάνιση tags ολοκληρωμένων μαθημάτων και επιλογή ομαδικού καθαρισμού

3.5 Panel πληροφοριών μαθήματος

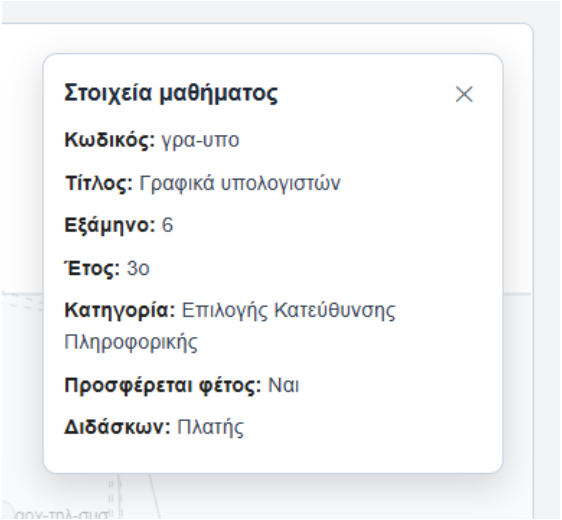
Αναλυτικές πληροφορίες εμφανίζονται μέσω του panel πληροφοριών για το κάθε μάθημα. Το panel εμφανίζεται όταν ο χρήστης πατήσει διπλό κλικ πάνω σε έναν κόμβο. Οι πληροφορίες που εμφανίζονται για το κάθε μάθημα είναι:

- ο κωδικός του μαθήματος
- ο πλήρης τίτλος του μαθήματος
- το εξαμήνο
- το έτος
- η κατηγορία του μαθήματος
- αν δικάζεται φέτος
- το όνομα του διδάσκων

Η λειτουργία αυτή επιτρέπει στο γράφημα να παραμένει οπτικά καθαρό, ενώ παράλληλα προσφέρει αναλυτικές πληροφορίες όταν επιλέξει ο χρήστης να εμφανιστούν.



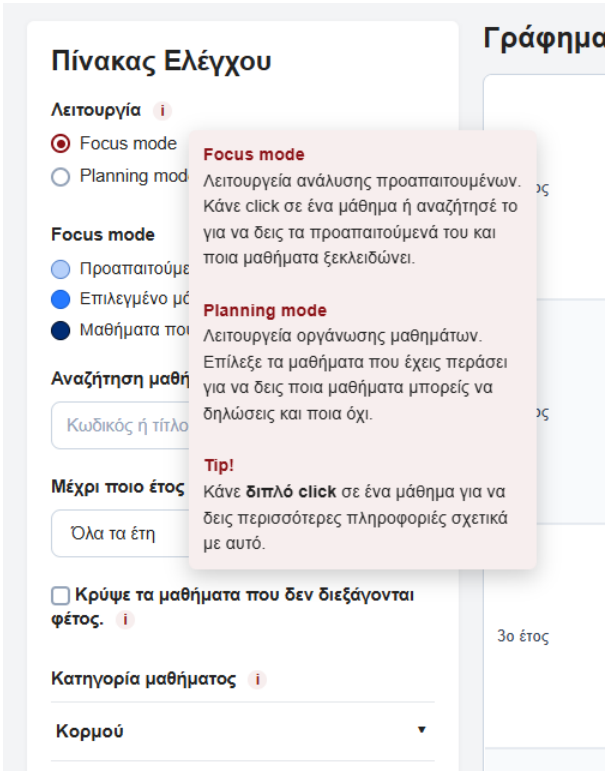
Γράφος με ενεργό panel πληροφοριών μαθήματος



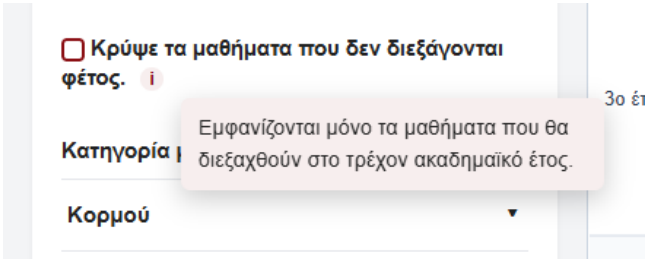
Panel πληροφοριών μαθήματος

3.6 Tooltips

Η εφαρμογής περιλαμβάνει tooltips τα οποία παρέχουν επιπλέον επεξηγηματικές πληροφορίες όταν ο χρήστης τοποθετεί τον δείκτη του ποντικιού πάνω στα αντίστοιχα εικονίδια πληροφοριών (info icons).



Tooltip επεξήγησης των βασικών λειτουργιών



Tooltip φίλτρου ακαδημαϊκού έτους

Κεφάλαιο 4

Βάση δεδομένων και τεχνολογίες υλοποίησης

Σε αυτό το κεφάλαιο παρουσιάζεται η βάση δεδομένων και οι τεχνολογίες, οι γλώσσες και οι βιβλιοθήκες που χρησιμοποιήθηκαν για την υλοποίηση της εφαρμογής. Πιο συγκεκριμένα, περιγράφεται αναλυτικά η ακαδημαϊκή πληροφορία που χρησιμοποιήθηκε από το σύστημα, οι μηχανισμοί που είναι υπεύθυνοι για την επιστροφή και επεξεργασία των δεδομένων και οι τεχνολογίες που χρησιμοποιήθηκαν για την δημιουργία του συστήματος.

4.1 Βάση δεδομένων

Η εφαρμογή χρησιμοποιεί SQLite βάση δεδομένων η οποία παρασχέθηκε από το τμήμα Πληροφορικής και Τηλεπικοινωνίων του πανεπιστήμιου. Η βάση περιέχει την ίδια ακαδημαϊκή πληροφορία που χρησιμοποιείται στον επίσημο οδηγό σπουδών του τμήματος.

Η εφαρμογή δέχεται πληροφορίες για τα μαθήματα, τις σχέσεις των προαπαιτούμενων, τις κατηγορίες των μαθήματων, και την διαθεσιμότητα του κάθε μαθήματος από την βάση έτσι ώστε να δημιουργηθεί το οπτικό γράφημα των προαπαιτούμενων.

Οι κύριοι πίνακες που χρησιμοποιήθηκαν από την εφαρμογή είναι:

```
lesson
prereq2lesson
teacher2lesson
```

Ο πίνακας `lesson` περιέχει την κυριά πληροφορία που σχετίζεται με το κάθε μάθημα όπως:

```
lesson_code
title
semester
cat_id
```

Το πεδίο `lesson_code` χρησιμοποιείται ως μοναδικό αναγνωριστικό για κάθε μάθημα.

Το πεδίο `title` περιεχει τον τίτλο του κάθε μαθήματος.

Το πεδίο semester ως μοναδικό περιέχει το εξάμηνο που διεξάγεται το κάθε μάθημα και χρησιμοποιείται για τον υπολογισμό του έτους.

Το cat_id περιέχει την κατηγορία του κάθε μαθήματος και χρησιμοποιείται από τον μηχανισμό του φιλτραρίσματος.

Ο πίνακας prereq2lesson περιέχει τις σχέσεις των προαπαιτούμενων μεταξύ των μαθημάτων.

Τα πεδία που χρησιμοποιούνται από αυτόν τον πίνακα είναι:

lesson_code
requirement_id
group_id

Το πεδίο lesson_code αντιπροσωπεύει το μάθημα που έχει προαπαιτούμενα.

Το πεδίο requirement_id αντιπροσωπεύει το μάθημα που είναι προαπαιτούμενο για το lesson_code.

Το πεδίο group_id χρησιμοποιείται για την διάκριση των προαπαιτούμενων ομάδων και τον προσδιορισμό του εάν οι προαπαιτούμενες συνδέσεις είναι σχέσεις AND (1) ή OR (0).

Για παράδειγμα, το μάθημα Γραφικά Υπολογιστών (γρα-υπο) έχει τις παρακάτω εγγραφές στον πίνακα prereq2lesson.

Πίνακας 8: τμήμα του πίνακα prereq2lesson

lesson_code	requirement_id	group_id
γρα-υπο	προ-2	0
γρα-υπο	αντ-προ	0
γρα-υπο	μαθ-1	1

Με βάση αυτές τις εγγραφές προκύπτει ότι ο φοιτητής πρέπει να έχει ολοκληρώσει υποχρεωτικά το μάθημα Μαθηματικά Ι (μαθ-1), καθώς ανήκει στην ομάδα group_id = 1 (AND). Παράλληλα, απαιτείται η ολοκλήρωση ενός από τα μαθήματα Προγραμματισμός ΙΙ (προ-2) ή Αντικειμενοστρεφής Προγραμματισμός (αντ-προ), τα οποία ανήκουν στην ομάδα group_id = 0 (OR).

Ο πίνακας `teacher2lesson` χρησιμοποιείται για να καθορίσει εάν το μάθημα διεξάγεται το τρέχον ακαδημαϊκό έτος. Όπως έχει αναφερθεί, η εφαρμογή ελέγχει εάν κάποιος καθηγητής έχει ανατεθεί στο κάθε μάθημα, και χρησιμοποίει αυτή την πληροφορία για να φιλτράρει τα μαθήματα με βάση την διαθεσιμότητά τους.

Η επικοινωνία με την βάση δεδομένων επιτυγχάνεται μέσω PHP Data Objects (PDO).

4.2 Γλώσσες υλοποίησης

4.2.1 PHP

Η PHP είναι μια γλώσσα προγραμματισμού που χρησιμοποιείται ευρέως στην ανάπτυξη web εφαρμογών. Η PHP εκτελείται στον διακομιστή (server) και χρησιμοποιείται κυρίως για την διαχείριση δεδομένων, την δημιουργία δυναμικών ιστοσελίδων και την επικοινωνία μεταξύ των ιστοσελίδων με τις βάσεις δεδομένων. [6]

Συγκεκριμένα στην εφαρμογή μας, η PHP χρησιμοποιείται για την ανάκτηση των ακαδημαϊκών δεδομένων από τη βάση δεδομένων SQLite και για την μετατροπή τους σε μορφή JSON, ώστε να μπορούν να αξιοποιηθούν από το σύστημα οπτικοποίησης στο frontend.

Η βασική λειτουργία του backend υλοποιείται στο αρχείο `graph-data.php`, το οποίο αναλαμβάνει να διαβάσει τα δεδομένα των μαθημάτων και των προαπαιτούμενων σχέσεων από την βάση, να τα επεξεργαστεί και να δημιουργήσει την δομή του γράφου που εμφανίζεται στην εφαρμογή.

Για την εργασία μας, επιλέξαμε την PHP επειδή προσφέρει έναν απλό και αποδοτικό τρόπο επικοινωνίας μεταξύ της βάσης δεδομένων και της frontend εφαρμογής. Έτσι διευκολύνεται η διαχείριση και μεταφορά των δεδομένων.

4.2.2 HTML

Η HTML (HyperText Markup Language) αποτελεί τη βασική γλώσσα σήμανσης για τη δημιουργία και τη δομή ιστοσελίδων. Η HTML είναι μία γλώσσα περιγραφής που βασίζεται σε χαρακτήρες απλού κειμένου και έχει σκοπό να περιγράφει τη δομή και τις ιδιότητες των αντικειμένων των εγγράφων υπερκειμένου. [7]

Στην παρούσα εφαρμογή, η HTML χρησιμοποιήθηκε για την κατασκευή του γραφικού περιβάλλοντος χρήστη (header, sidebar, περιοχή εμφάνισης του γράφου, φίλτρα, μπάρα αναζήτησης, μηχανισμό φιλτραρίσματος, επεξηγήσεις/ legends, tooltips, κουμπιά, μενού

accordion, πάνελ πληροφοριών μαθημάτων και, γενικότερα, των διαδραστικών στοιχείων της εφαρμογής).

4.2.3 CSS

Η CSS (Cascading Style Sheets) είναι η γλώσσα που χρησιμοποιείται για τον καθορισμό της εμφάνισης και της διάταξης μιας ιστοσελίδας. [8]

Στο σύστημα μας χρησιμοποιήσαμε την CSS για τη μορφοποίηση του γραφικού περιβάλλοντος χρήστη (χρώματα, τυπογραφία, αποστάσεις, στοίχιση των στοιχείων, legends, tooltips, hover effects, animations, κουμπιά και, γενικότερα, τα οπτικά στοιχεία που σχετίζονται με τον γράφο και την διάταξη της εφαρμογής). Ιδιαίτερη έμφαση δόθηκε στη χρηστικότητα και στην οπτική καθαρότητα, ώστε η αλληλεπίδραση του χρήστη με τον γράφο να είναι πιο εύκολη και κατανοητή. Τέλος, αξίζει να σημειωθεί ότι για τη μορφοποίηση της εφαρμογής δεν χρησιμοποιήθηκαν εξωτερικές βιβλιοθήκες CSS.

4.2.4 JavaScript

Η JavaScript είναι μια από τις πιο βασικές και διαδεδομένες γλώσσες προγραμματισμού, για την ανάπτυξη διαδραστικών web εφαρμογών. Ο συνδυασμός της μαζί με την HTML συμβάλλει στην δημιουργία δυναμικών ιστοσελίδων, δηλαδή ιστοσελίδων που περιέχουν δυναμικό περιεχόμενο. Με την χρήση της μπορούμε να υλοποιήσουμε μια εφαρμογή που να αντιδρά σε γεγονότα που μπορούν να συμβούν σε αυτή, καθώς και να τροποποιούμε δεδομένα. [9]

Στην εφαρμογή μας, χρησιμοποιήσαμε την JavaScript για την υλοποίηση όλων των διαδραστικών και δυναμικών λειτουργιών της εφαρμογής. Μερικές από αυτές τις λειτουργίες είναι η αλληλεπίδραση με τον γράφο, η δυναμική ενημέρωση των δεδομένων, η αναζήτηση μαθημάτων, οι μηχανισμοί φιλτραρίσματος, η διαχείριση των tooltips, οι αλλαγές στις οπτικές καταστάσεις, καθώς και τις λειτουργίες του focus mode, του planning mode και του drag selection.

4.3 Βιβλιοθήκες και εργαλεία

4.3.1 D3.js

Η D3.js είναι μια βιβλιοθήκη της JavaScript, που χρησιμοποιείται για τη δημιουργία δυναμικών και διαδραστικών οπτικοποιήσεων δεδομένων με τη χρήση τεχνολογιών όπως SVG, HTML και CSS.

Επιλέξαμε την συγκεκριμένη βιβλιοθήκη επειδή παρέχει μεγάλη ευελιξία στη δημιουργία γραφημάτων και υποστηρίζει δυναμική αλληλεπίδραση με στοιχεία που βασίζονται σε δεδομένα. [4]

Πιο συγκεκριμένα, η D3.js συμβάλει στις εξής ενέργειες:

- Δημιουργία SVG γράφου
- Εμφάνιση των κόμβων και απεικόνιση των συνδέσεων προαπαιτούμενων
- Δυναμική τοποθέτηση των κόμβων και προσομοίωση δυνάμεων (force simulation)
- Δυναμική ενημέρωση του γράφου

Τα SVG (Scalable Vector Graphics) στοιχεία είναι γραφικά διανυσματικού τύπου που χρησιμοποιούνται για την εμφάνιση σχημάτων, γραμμών και γενικότερα γραφικών μέσα σε μια ιστοσελίδα. [2]

Σε αντίθεση με τις κλασικές εικόνες (όπως το JPEG και το PNG) που αποτελούνται από pixels, το SVG χρησιμοποιεί μαθηματικούς τύπους και συντεταγμένες, και όχι pixels. Έτσι, τα SVG αρχεία επιτυγχάνουν να διατηρούν την ποιότητα της εικόνας ακόμα κι όταν γίνεται μεγέθυνση ή σμίκρυνση. [2]

4.3.2 PDO

Το PDO (PHP Data Objects) είναι μια επέκταση της PHP μέσω της οποίας δημιουργείται ένας ενιαίος και ασφαλής τρόπο επικοινωνίας με βάσεις δεδομένων. [10]

Στην συγκεκριμένη εφαρμογή, το PDO χρησιμοποιείται για τη σύνδεση και την αλληλεπίδραση της PHP με τη βάση δεδομένων SQLite.

Επιλέξαμε το PDO έγινε επειδή κάνει την διαχείριση της βάσης δεδομένων πιο απλή.

4.3.3 SQLite

Η SQLite είναι ένα σύστημα διαχείρισης σχεσιακών βάσεων δεδομένων η οποία αποθηκεύει όλη την βάση σε ένα αρχείο. Ξεχωρίζει για το πόσο ελαφρύ είναι σαν σύστημα. [1]

Επιλέξαμε την SQLite επειδή η υπάρχουσα βάση που δόθηκε από το τμήμα του πανεπιστήμιου ήταν ήδη υλοποιημένη σε αυτή την μορφή. Για αυτόν τον λόγο, αποφασίσαμε να διατηρήσουμε την ίδια τεχνολογία, ώστε να εξασφαλιστεί η συμβατότητα με τα διαθέσιμα δεδομένα των μαθήματων.

Η SQLite δουλεύει μαζί με την PHP και το PDO για την ανάκτηση των ακαδημαϊκών δεδομένων που χρησιμοποιούνται από την εφαρμογή.

4.3.4 JSON

Το JSON (JavaScript Object Notation) είναι μια ελαφριά μορφή ανταλλαγής δεδομένων, η οποία χρησιμοποιείται ευρέως σε web εφαρμογές για την επικοινωνία μεταξύ backend και frontend συστημάτων. [5]

Στην εφαρμογή μας, το JSON χρησιμοποιείται για την μεταφορά των δεδομένων του γραφήματος μεταξύ της PHP του backend και της D3.js του συστήματος οπτικοποίησης.

Κεφάλαιο 5

Αρχιτεκτονική συστήματος και υλοποίηση

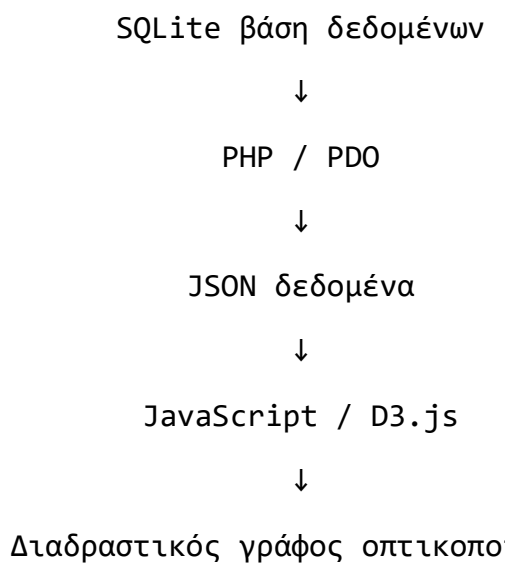
Σε αυτό το κεφάλαιο, περιγράφεται η οργάνωση του συστήματος και η δομή του κώδικα. Ακόμα, αναλύεται ο τρόπος λειτουργίας του οπτικού γράφου, των διάφορων μηχανισμών και των δυο κύριων λειτουργιών αλληλεπίδρασης με τον γράφο, το focus mode και το planning mode.

5.1 Αρχιτεκτονική συστήματος

Η παρούσα εφαρμογή στηρίζεται στην αρχιτεκτονική του client-server.

Το backend τμήμα ανακτά τα ακαδημαϊκά δεδομένα από την SQLite βάση δεδομένων και τα μετατρέπει σε μορφή JSON. Το frontend τμήμα δέχεται αυτά τα δεδομένα και τα χρησιμοποιεί για να δημιουργήσει τον γράφο οπτικοποίησης.

Η ροή του συστήματος φαίνεται στο παρακάτω σχέδιο:



5.2 Σύνδεση με τη βάση και ανάκτηση δεδομένων

Η ανάκτηση των δεδομένων πραγματοποιείται από το αρχείο `graph-data.php`. Η επικοινωνία με τη βάση δεδομένων επιτυγχάνεται μέσω PHP Data Objects (PDO).

Η σύνδεση της SQLite βάσης γίνεται με τον ακόλουθο κώδικα:

```
<?php
try {
    $db = new PDO("sqlite:lessons.sqlite");
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}
?>
```

Σύνδεση SQLite database

Στη συνέχεια, ο ακόλουθος κώδικας ανακτά τις σχέσεις των προαπαιτούμενων από τη βάση δεδομένων:

```
$stmt = $db->prepare("
  SELECT lesson_code, requirement_id, group_id
  FROM prereq2lesson
");
```

Οι εγγραφές που επιστρέφονται περιλαμβάνουν τον κωδικό του μαθήματος (`lesson_code`), το αντίστοιχο προαπαιτούμενο (`requirement_id`) και το `group_id`, το οποίο χρησιμοποιείται για τον προσδιορισμό των σχέσεων AND και OR μεταξύ των προαπαιτούμενων.

Στη συνέχεια, οι σχέσεις αυτές μετατρέπονται σε συνδέσεις (links) που χρησιμοποιούνται από τον γράφο:

```
$links[] = [
   "source" => $r["requirement_id"],
   "target" => $r["lesson_code"],
   "type"   => $type,
   "group"  => (int)$r["group_id"]
];
```

Αφού ολοκληρωθεί η επεξεργασία, το backend τα μετατρέπει σε μορφή JSON. Η δομή JSON περιέχει δύο βασικές συλλογές:

```
nodes
links
```

Η συλλογή `nodes` αποθηκεύει τους κόμβους του γράφου που αντιπροσωπεύουν τα μαθήματα, ενώ η συλλογή `links` αποθηκεύει τις σχέσεις προαπαιτούμενων μεταξύ των μαθημάτων.

Το τελικό JSON δημιουργείται με τον ακόλουθο κώδικα:

```
echo json_encode([
    "nodes" => $nodes,
    "links" => $links
], JSON_UNESCAPED_UNICODE);
```

Τελική απάντηση JSON

Η τελική απάντηση στέλνεται στο frontend και χρησιμοποιείται για τη δημιουργία της οπτικοποίησης του γράφου. Η διαδικασία αυτή περιγράφεται στην Ενότητα 5.3

5.3 Φόρτωση δεδομένων γράφου

Τα δεδομένα του γράφου φορτώνονται δυναμικά από το αρχείο `graph-data.php` με τον ακόλουθο κώδικα:

```
fetch("graph-data.php")
  .then(r => r.json())
  .then(data => {
    graphData = data;
    renderGraph(data);
  })
  .catch(err => {
    console.error(err);
    d3.select("#graph").append("div").text("Failed to load graph-data.php");
  });
```

Τα δεδομένα φορτώνονται χρησιμοποιώντας την συνάρτηση `fetch()`. Το αίτημα στέλνεται στο αρχείο `graph-data.php`, το οποίο επιστρέφει τα ακαδημαϊκά δεδομένα σε μορφή JSON. Μόλις μετατραπούν σε αντικείμενο της JavaScript, τα δεδομένα αποθηκεύονται στην μεταβλητή `graphData` και περνούν στην συνάρτηση `renderGraph()`.

Η `renderGraph()` είναι υπεύθυνη για την δημιουργία του γράφου.

Για την εξασφάλιση την ομαλής λειτουργίας, σε περίπτωση που εμφανιστεί κάποιο σφάλμα κατά την διάρκεια της φόρτωσης, εμφανίζεται το μήνυμα «`Failed to load graph-data.php`» στον browser.

5.4 Δημιουργία γράφου

Ο γράφος εμφανίζεται μέσα στο HTML στοιχείο `<div id="graph"></div>`.
Η εφαρμογή χρησιμοποιεί SVG στοιχεία τα οποία μέσω της βιβλιοθήκης D3.js δημιουργούνται δυναμικά.

Μέσα στην συνάρτηση `rednerGraph()`, το SVG πλαίσιο δημιουργείται με τον ακόλουθο κώδικα:

```
const svg = d3.select("#graph")
    .append("svg")
    .attr("viewBox", `0 0 ${width} ${height}`)
    .attr("preserveAspectRatio", "xMidYMid meet")
    .style("width", "100%")
    .style("height", "100%");
```

Το SVG αποτελεί το βασικό πλαίσιο μέσα στο οποίο δημιουργούνται και τοποθετούνται τα στοιχεία της οπτικοποίησης. Περισσότερες λεπτομέρειες για τη διαδικασία τοποθέτησης και κατανομής των κόμβων παρουσιάζονται στην Ενότητα 5.6

5.5 Προετοιμασία κόμβων και συνδέσεων

Η εφαρμογή επεξεργάζεται τα δεδομένα που δέχεται πριν γίνει η οπτικοποίηση του γράφου. Αυτό συμβαίνει επειδή ορισμένα μαθήματα που δεν σχετίζονται με την ανάλυση των σχέσεων προαπαιτούμενων, τα θεωρούμε μη απαραίτητα να εμφανιστούν στο γράφημα. Έτσι το σύστημα ορισμένα μαθήματα τα εξαιρεί από την απεικόνιση.

Πιο συγκεκριμένα, τα ids των μαθημάτων που εξαιρούνται αποθηκεύονται στην μεταβλητή `NOT_INCLUDED_IDS` μέσω του παρακάτω κώδικα:

```
const EXCLUDED_IDS = new Set([
  "ξεν-γλω",
  "πρα-ασκ-ερα",
  "πρα-ασκ-πδε",
  "πτυ-εργ-2"
]);
```

Έπειτα, οι κόμβοι φιλτράρονται ώστε να αφαιρεθούν τα επιλεγμένα μαθήματα:

```
nodes = nodes.filter(n => !NOT_INCLUDED_IDS.has(String(n.id ||
"").toLowerCase()));
```

Αφού ολοκληρωθεί το φιλτράρισμα των κόμβων, φιλτράρονται και οι συνδέσεις του γράφου. Έτσι, παραμένουν μόνο οι σχέσεις μεταξύ μαθημάτων που εμφανίζονται στην οπτικοποίηση.

```
const validNodeIds = new Set(nodes.map(n => n.id));


  links = links.filter(l => {
    const sourceId = String(l.source || "").toLowerCase();
    const targetId = String(l.target || "").toLowerCase();


    return (
      validNodeIds.has(l.source) &&
      validNodeIds.has(l.target) &&
      !NOT_INCLUDED_IDS.has(sourceId) &&
      !NOT_INCLUDED_IDS.has(targetId)
    );
  });
```

Παράλληλα, τα ids μετατρέπονται σε πεζά γράμματα ώστε να εξασφαλίζεται η ομοιομορφία κατά τη σύγκριση και επεξεργασία τους.

5.6 Προσομοίωση γράφου (Graph Simulation)

5.6.1 Force Simulation

Το simulation είναι υπεύθυνο για την δυναμική τοποθέτηση των κόμβων στον χώρο. Μέσω του μηχανισμού force simulation της D3.js υλοποιείται η διάταξη του γράφου. Ο μηχανισμός αυτός τοποθετεί τους κόμβους αυτόματα στον χώρο εφαρμόζοντας διαφορετικές δυνάμεις (έλξης, απώθησης και σύγκρουσης). Κατά την έναρξη του simulation οι κόμβοι κινούνται συνεχώς μέχρι να πραγματοποιηθεί μία ισορροπημένη διάταξη. Η ένταση και η διάρκεια αυτής της κίνησης εξαρτώνται από τις παραμέτρους που έχουν οριστεί (η απόσταση μεταξύ συνδεδεμένων κόμβων, η δύναμη απώθησης και ο ρυθμός με τον οποίο το simulation σταθεροποιείται). Η κίνηση συνεχίζεται μέχρι οι δυνάμεις να σταθεροποιηθούν και ο γράφος να καταλήξει σε μία σταθερή και ευανάγνωστη διάταξη.

Δόθηκε ιδιαίτερη έμφαση στην τοποθέτηση των κόμβων, τόσο στο να απλώνονται ομοιόμορφα στον χώρο, όσο και στην αποφυγή επικαλύψεων μεταξύ των κόμβων, δηλαδή ο ένα κόμβος να μην πέφτει πάνω στον άλλον.

Η υλοποίηση του simulation γίνεται με τον ακόλουθο κώδικα:

```
const simulation = d3.forceSimulation(nodes)
    .alpha(0.7)
    .alphaDecay(0.08)
    .force("link", d3.forceLink(validLinks).id(d => d.id).distance(190))
    .force("charge", d3.forceManyBody().strength(-320))
    .force("x", d3.forceX(d => d.xTarget).strength(0.008))
    .force("collide", d3.forceCollide(nodeSize + nodePadding));
```

Όπως φαίνεται από το τμήμα κώδικα, το γράφημα χρησιμοποιεί διαφορετικούς τύπους δυνάμεων (forces), όπου η κάθε δύναμη επηρεάζει διαφορετικά τους κόμβους.

- Link Force

Δύναμη σύνδεσης. Διατηρεί κοντά μεταξύ τους τους κόμβους που συνδέονται με συνδέσεις. [3]

- Charge Force

Δύναμη απώθησης. Απομακρύνει τους κόμβους μεταξύ τους σε απόσταση, μειώνοντας τις επικαλύψεις και κάνοντας τον γράφο πιο ευανάγνωστο. [3]

- X Position Force

Δύναμη τοποθέτησης στον άξονα x. Οργανώνει τους κόμβους οριζόντια, ανάλογα με το ακαδημαϊκό έτος στο οποίο ανήκουν. [3]

- Collision Force

Δύναμη σύγκρουσης. Αποτρέπει την οπτική επικάλυψη των κόμβων, ελέγχοντας το μέγεθος και το περιθώριο απόστασης των κόμβων, χωρίς να τους αφήνει να πέσει ο ένας κόμβος πάνω στον άλλον. [3]

Επιπρόσθετα, οι παράμετροι `alpha` και `alphaDecay` ελέγχουν την συμπεριφορά του simulation. Η παράμετρος alpha καθορίζει την ένταση της αρχικής κίνησης των κόμβων. Η `alphaDecay` καθορίζει τον ρυθμό με τον οποίο το simulation σταθεροποιείται με βάση τον χρόνο.

5.6.2 Περιορισμός και κατανομή κόμβων ανά ακαδημαϊκό έτος

Πριν από την έναρξη του simulation, κάθε μάθημα, με βάση το εξάμηνό του, αντιστοιχίζεται στο ακαδημαϊκό έτος στο οποίο ανήκει. Παράλληλα, υπολογίζεται μία αρχική κατακόρυφη θέση για κάθε κόμβο μέσα στο πλαίσιο του αντίστοιχου έτους

```
nodes.forEach(d => {
  d.year = Math.ceil(d.semester / 2);

  const top = (d.year - 1) * heightRow + 20;
  const bottom = d.year * heightRow - 20;

  d.yTarget = top + Math.random() * (bottom - top);
  d.y = d.yTarget;
  d.fy = d.yTarget;
```

```
  });
```

Η μεταβλητή `d.year` προκύπτει από το εξάμηνο του μαθήματος, ενώ οι μεταβλητές `top` και `bottom` ορίζουν τα κατακόρυφα όρια του αντίστοιχου ακαδημαϊκού έτους. Η `fy` διατηρεί σταθερή την κατακόρυφη θέση κάθε κόμβου, ώστε να παραμένει στην περιοχή του έτους στο οποίο ανήκει.

Όσον αφορά την οριζόντια κατανομή των κόμβων, οι κόμβοι ομαδοποιούνται ανά ακαδημαϊκό έτος και υπολογίζεται η θέση `xTarget` για κάθε κόμβο.

```
const nodesGroupYear = d3.group(nodes, d => d.year);


nodesGroupYear.forEach(nodeYears => {


   const left = 260;
   const right = width - 360;
   const availableWidth = right - left;


   nodeYears.forEach((d, i) => {
     const t = nodeYears.length === 1 ? 0.5 : i / (nodeYears.length -
1);
     const moveShake = (Math.random() - 0.5) * 80;


     d.xTarget = left + t * availableWidth + moveShake;
     d.x = d.xTarget;
   });
 });
```

Κάθε κόμβος έλκεται προς την θέση `xTarget` μέσω της δύναμης `forceX`, με αποτέλεσμα οι κόμβοι να παραμένουν στο αντίστοιχο ακαδημαϊκό έτος.

5.6.3 Ενημέρωση θέσεων κόμβων (Tick Function)

Οι θέσεις των στοιχείων του γράφου κατά την διάρκεια του simulation συνεχώς ενημερώνονται μέσω του `tick`:

```
simulation.on("tick", () => {
    nodes.forEach(d => {
      d.x = Math.max(220, Math.min(width - 260, d.x));
    });

    link
      .attr("x1", d => d.source.x)
      .attr("y1", d => d.source.y)
      .attr("x2", d => d.target.x)
      .attr("y2", d => d.target.y);

    node
      .attr("cx", d => d.x)
      .attr("cy", d => d.y);

    label
      .attr("x", d => d.x)
      .attr("y", d => d.y);

    if (simulation.alpha() < 0.03) simulation.stop();
  });
```

Οι θέσεις των κόμβων, των συνδέσεων και των ετικετών του γράφου ενημερώνεται δυναμικά σε κάθε σημείο του simulation. Για να παραμένουν οι κομβόι σε ορατή περιοχή εντός της εφαρμογής, παράλληλα εφαρμόζονται όρια στον οριζόντιο άξονα. Τέλος, όταν η τιμή `alpha`

γίνει πολύ μικρή, η εφαρμογή θεωρεί ότι ο γράφος έχει σταθεροποιηθεί και τερματίζει αυτόματα το simulation.

5.7 Μηχανισμοί φιλτραρίσματος

Τα φίλτρα που περιλαμβάνονται στην εφαρμογή αφορούν:

- Το ακαδημαϊκό έτος
- Την κατηγορία μαθήματος
- Την διαθεσιμότητα του μαθήματος

5.7.1 Ακαδημαϊκό έτος

Η συνάρτηση φιλτραρίσματος για το ακαδημαϊκό έτος είναι η `passesYear()`. Η συγκεκριμένη συνάρτηση ελέγχει αν το μάθημα ανήκει σε ακαδημαϊκό έτος μικρότερο ή ίσο από το όριο που έχει επιλεχτεί. Αν δεν έχει επιλεχτεί όριο για το έτος, τότε όλα τα μαθήματα θεωρούνται ορατά από default.

5.7.2 Κατηγορία μαθήματος

Η συνάρτηση φιλτραρίσματος για την κατηγορία του μαθήματος είναι η `passesCategory()`. Η συγκεκριμένη συνάρτηση ελέγχει αν η κατηγόρια του μαθήματος είναι ενεργοποιημένη στον πίνακα ελέγχου την εφαρμογής. Έτσι, με βάση το UI (User Interface) γίνεται αντιστοιχία για το ποια μαθήματα επιτρέπονται να εμφανιστούν σύμφωνα με τις κατηγορίες που έχει επιλέξει ο χρήστης.

5.7.3 Διαθεσιμότητα μαθήματος

Η συνάρτηση φιλτραρίσματος για την διαθεσιμότητα του μαθήματος είναι η `passesOffered()`. Η συγκεκριμένη συνάρτηση ελέγχει αν ο χρήστης έχει ενεργοποιήσει το φίλτρο διαθεσιμότητας και αν ναι, εμφανίζει στο γράφο μόνο τα μαθήματα που προσφέρονται στο τρέχον ακαδημαϊκό έτος.

5.7.4 Συνδυασμός συναρτήσεων φιλτραρίσματος

Οι τρεις συναρτήσεις που περιγράψαμε συνδυάζονται σε μια συνάρτηση, την `passesAllFillters()`, η οποία ελέγχοντας όλα τα ενεργά φίλτρα, εξετάζει αν ο κόμβος θα εμφανιστεί στον γράφο.

Χάριν της δυναμικής ενημέρωσης του γράφου, κάθε φορά που ο χρήστης αλλάζει κάποια επιλογή φιλτραρίσματος, η οπτικοποίηση προσαρμόζεται άμεσα.

5.8 Μηχανισμός αναζήτησης

Το σύστημα περιλαμβάνει μηχανισμό αναζήτησης, ο οποίος επιτρέπει στον χρήστη να εντοπίσει γρήγορα μέσα στον γράφο ένα συγκεκριμένο μάθημα, είτε αναζητώντας το με τον κωδικό του, είτε με τον τίτλο του.

Μέσω της βοηθητικής συνάρτησης `simpleSearchText()` αρχικά μετατρέπει το κείμενο σε πεζά γράμματα, αφαιρώντας τυχόν παύλες, κενά, γραμμές, τέλειες, κλπ. Με αυτόν το τρόπο η αναζήτηση γίνεται πιο ευέλικτη χωρίς να επηρεάζεται από διάφορες μορφοποιήσεις.

Η λειτουργία της αναζήτησης ενεργοποιείται κάθε φορά που ο χρήστης πληκτρολογεί στο πεδίο αναζήτησης (`searchEl.oninput`). Η εφαρμογή, λαμβάνει το κείμενο εισόδου, το κανονικοποιεί όπως περιγράψαμε παραπάνω και στην συνέχεια πραγματοποιεί κάποιους επιπρόσθετους ελέγχους.

Στην αρχή προσπαθεί να βρει την ακριβή αντιστοίχιση με τον κωδικό κάποιου μαθήματος, και αν δεν βρεθεί το σύστημα αναζητά μαθήματα των οποίων ο κωδικός ή τίτλος ξεκινούν με το κείμενο που έχει αναζητηθεί. Τέλος, γίνεται έλεγχος μερικής αντιστοίχισης εντός των κωδικών και των τίτλων των μαθήματων.

Όταν το Focus Mode είναι ενεργοποιημένο, το μάθημα που αναζητείται αλλάζει χρώμα αυτόματα στον γράφο και εμφανίζονται κατευθείαν τα προαπαιτούμενα και τα μαθήματα που ξεκλειδώνει.

5.9 Focus Mode

Το Focus Mode αποτελεί το πρώτο βασικό mode αλληλεπίδρασης της εφαρμογής.

Σε αυτή την λειτουργία, ο χρήστης επιλεγεί ένα μάθημα, είτε πατώντας κλικ πάνω στον κόμβο, είτε αναζητώντας το μάθημα μέσω του μηχανισμού αναζήτησης, και το σύστημα του εμφανίζει δυναμικά:

- Το επιλεγμένο μάθημα
- Τα προαπαιτούμενα μαθήματά του
- Τα μαθήματα που έχουν αυτό ως προαπαιτούμενο

Στον γράφο, ο κόμβος του επιλεγμένου μαθήματος αλλάζει σε μπλε χρώμα, οι κομβόι των προαπαιτούμενων σε μπλε ανοιχτό, και μαθήματα που ξεκλειδώνονται σε μπλε σκούρο.

Για την αποθήκευση των σχέσεων προαπαιτούμενων χρησιμοποιούνται δύο δομές γράφου: Το `forward map` και το `reverse map`.

Το `forward map` αποθηκεύει τα μαθήματα που έχουν αυτό ως προαπαιτούμενο.

Το `reverse map` αποθηκεύει τα προαπαιτούμενα που απαιτούνται για την παρακολούθηση ενός μαθήματος.

Οι συνδέσεις του γράφου εισάγονται στις παραπάνω δομές μέσω της συνάρτησης:

```
function addEdge(map, a, b) {
  if (!map.has(a)) map.set(a, []);
  map.get(a).push(b);
}
```

Και έπειτα δημιουργείται η δομή του γράφου προαπαιτούμενων:

```
validLinks.forEach(l => {
    addEdge(forward, l.source, l.target);
    addEdge(reverse, l.target, l.source);
  });
```

Η συνάρτηση `traverse()` είναι υπεύθυνη για την διάσχιση του γράφου:

```
function traverse(startId, adjacencyMap) {

    const visited = new Set([startId]);
    const stack = [startId];

    while (stack.length) {
      const current = stack.pop();
      const next = adjacencyMap.get(current) || [];

      for (const n of next) {
        if (!visited.has(n)) {
          visited.add(n);
          stack.push(n);
        }
      }
    }
    return visited;
  }
```

Η συνάρτηση `traverse()` διαχύει τον γράφο χρησιμοποιώντας στοίβα και καταγράφει όλους τους συνδεδεμένους κόμβους που σχετίζονται με το επιλεγμένο μάθημα. Μέσω του συνόλου visited αποφεύγονται οι επαναλήψεις.

Τα προαπαιτούμενα και εξαρτώμενα μαθήματα υπολογίζονται μέσω της συνάρτησης `findRelevantCourses()`.

```
function findRelevantCourses(focusId) {
    const prereqs = traverse(focusId, reverse);
```

```
    const dependents = traverse(focusId, forward);
    const relevant = new Set([...prereqs, ...dependents]);
    return { prereqs, dependents, relevant };
  }
```

Πιο συγκεκριμένα, η συνάρτηση υπολογίζει τα προαπαιτούμενα του επιλεγμένου μαθήματος (`prereqs`) , τα μαθήματα που ξεκλειδώνονται από αυτό (`dependents`) και το συνολικό σύνολο των σχετικών μαθημάτων (`prereqs και dependents`)

Το σύστημα χρησιμοποιεί αυτά τα σύνολα για την δυναμική επισήμανση της δομής των προαπαιτούμενων εντός του γράφου.

Κατ' επέκταση, η εφαρμογή οργανώνει τις σχέσεις προαπαιτούμενων μέσω της δομής prereqGroups, πράγμα που χρησιμοποιείται στην δεύτερη βασική λειτουργία της εφαρμογής, στο Planning Mode. Στην επόμενη ενότητα, θα αναλύσουμε πώς μέσω αυτής της δομής αξιολογούνται οι συνθήκες προαπαιτούμενων και υπολογίζεται αν ένα μάθημα είναι διαθέσιμο με βάση τα μαθήματα που έχει ήδη ολοκληρώσει ο φοιτητής.

Τέλος, αξίζει να αναλύσουμε τις βοηθητικές συναρτήσεις του Focus Mode, την συνάρτηση applyFocus() και την συνάρτηση clearFocus().

```
  function applyFocus(focusId) {
    if (!nodeWithId.has(focusId)) return;
    currentFocusId = focusId;
    saveFocus();
    applyVisuals();
  }

function clearFocus() {
    currentFocusId = null;
    saveFocus();
    applyVisuals();
  }
```

Η applyFocus() χρησιμοποιείται για τον ορισμό ενός νέου focus μαθήματος στον γράφο, ενώ η clearFocus() χρησιμοποιείται για την αφαίρεση του τρέχοντος focus μαθήματος.

5.10 Planning Mode

Το Planning Mode αποτελεί το δεύτερο βασικό mode αλληλεπίδρασης της εφαρμογής.

Σε αυτή την λειτουργία, ο χρήστης επιλεγεί τα μαθήματα που έχει ήδη ολοκληρώση, είτε κάνοντας κλικ πάνω στους κόμβους, είτε σέρνοντας τον κέρσορα για την επιλογή πολλαπλών μαθημάτων (drag selection). Με βάση τα μαθήματα που έχει επιλέξει ο χρήστης, το σύστημα εξετάζει τις σχέσεις προαπαιτούμενων και υπολογίζει ποια μαθήματα είναι διαθέσιμα προς παρακολούθηση και ποια όχι.

Στον γράφο, οι κόμβοι των διαθεσίμων μαθημάτων αλλάζουν σε πράσινο χρώμα, οι κόμβοι των μη διαθέσιμων μαθήματων σε κόκκινο, και τα επιλεγμένα μαθήματα σε μπλε.

Όπως αναφέραμε συνοπτικά στην Ενότητα 5.9, οι σχέσεις προαπαιτούμενων οργανώνονται σε ομάδες μέσω της δομής prereqGroups:

```
const prereqGroups = new Map();

  validLinks.forEach(l => {

    const target = l.target;
    const source = l.source;
    const group = l.group ?? 0;

    if (!prereqGroups.has(target)) prereqGroups.set(target, new Map());
    const targetGroups = prereqGroups.get(target);

    if (!targetGroups.has(group)) targetGroups.set(group, []);
    targetGroups.get(group).push(source);
```

```
  });
```

Η δομή prereqGroups αποθηκεύει τα προαπαιτούμενα κάθε μαθήματος οργανωμένα σε ομάδες. Η μεταβλητή `target` δηλώνει το μάθημα που έχει προαπαιτούμενα, ενώ η μεταβλητή `source` δηλώνει ένα προαπαιτούμενο μάθημα. Η μεταβλητή `group` χρησιμοποιείται για τον διαχωρισμό των προϋποθέσεων σε ανεξάρτητες ομάδες, τύπου AND και OR.

Τα ολοκληρωμένα μαθήματα αποθηκεύονται στο σύνολο `passedCourses`.

```
const passedCourses = new Set(window.__passedCourses || []);
```

Τα οποία διατηρούνται μέσω της συνάρτησης `saveFocus()`, επιτρέποντας την διατήρηση της κατάστασης της εφαρμογής, σε τυχόν αλλαγές του φιλτραρίσματος.

Η διαθεσιμότητα ενός μαθήματος υπολογίζεται μέσω της συνάρτησης `checkCourseAvailable()`:

```
function checkCourseAvailable(courseId) {
    if (passedCourses.has(courseId)) return false;

    const groups = prereqGroups.get(courseId);

    if (!groups || groups.size === 0) return true;

    for (const reqList of groups.values()) {
      const groupSatisfied = reqList.some(reqId => passedCourses.has(reqId));
      if (!groupSatisfied) return false;
    }

    return true;
  }
```

Αρχικά ελέγχεται αν το μάθημα έχει ήδη ολοκληρωθεί από τον χρήστη. Αν το μάθημα υπάρχει ήδη στο passedCourses, τότε θεωρείται μη διαθέσιμο προς επιλογή.

Στη συνέχεια, το σύστημα ανακτά τις ομάδες προαπαιτούμενων του μαθήματος. Αν δεν υπάρχουν προαπαιτούμενα, το μάθημα θεωρείται άμεσα διαθέσιμο. Αν υπάρχουν ομάδες προϋποθέσεων, κάθε ομάδα εξετάζεται ξεχωριστά. Μια ομάδα θεωρείται ικανοποιημένη όταν τουλάχιστον ένα από τα μαθήματα της ομάδας έχει ολοκληρωθεί από τον χρήστη. Αν ικανοποιούνται όλες οι ομάδες, το μάθημα θεωρείται διαθέσιμο.

Τα ολοκληρωμένα μαθήματα του χρήστη εμφανίζονται σε μορφή tags στον Πίνακα Ελέγχου της εφαρμογής. Αυτό επιτυγχάνεται μέσω της συνάρτησης giveTagsPassedCourses():

```
function giveTagsPassedCourses() {
    const passedListEl = document.getElementById("passed-list");
    if (!passedListEl) return;

    passedListEl.innerHTML = "";

    if (passedCourses.size === 0) {
      passedListEl.innerHTML = '<div class="helper-text">Δεν έχουν επιλεγεί περασμένα μαθήματα.</div>';
      return;
    }

    [...passedCourses].sort().forEach(id => {
      const tag = document.createElement("button");
      tag.type = "button";
      tag.className = "passed-tag";
      tag.textContent = `${id} ✕`;
      tag.onclick = () => {
        passedCourses.delete(id);
        saveFocus();
```

```
        giveTagsPassedCourses();
        applyVisuals();
      };
      passedListEl.appendChild(tag);
    });
  }
```

Η συνάρτηση δημιουργεί tags για όλα τα ολοκληρωμένα μαθήματα του χρήστη. Κάθε tag εμφανίζει το id του μαθήματος δίνεται η επιλογή στον χρήστη να αφαιρέσει κάποιο επιλεγμένο μάθημα σε περίπτωση λανθασμένης επιλογής. Η εφαρμογή εμφανίζει σχετικό ενημερωτικό μήνυμα, αν δεν υπάρχουν επιλεγμένα μαθήματα.

Η προσθήκη και αφαίρεση ολοκληρωμένων μαθημάτων πραγματοποιείται μέσω της συνάρτησης `adjustPassedCourse()`:

```
function adjustPassedCourse(courseId) {
    if (passedCourses.has(courseId)) passedCourses.delete(courseId);
    else passedCourses.add(courseId);

    saveFocus();
    giveTagsPassedCourses();
    applyVisuals();
  }
```

Η συνάρτηση είναι υπεύθυνη στην προσθήκη και αφαίρεση μαθήματων από το σύνολο των ολοκληρωμένων μαθημάτων. Σε κάθε αλλαγή που πραγματοποιείται, το σύστημα αποθηκεύει την νέα κατάσταση, ενημερώνει τα tags και ανανεώνει δυναμικά την οπτικοποίηση του γράφου.

5.10.1 Μηχανισμός Drag Selection

Το Planning Mode υποστηρίζει επίσης λειτουργία επιλογής με σύρσιμο του κέρσορα, η οποία επιτρέπει στον χρήστη να επιλεγεί πολλά ολοκληρωμένα μαθήματα μαζί.

Μέσω της βοηθητικής συνάρτησης `getPointerPosition()`, η θέση του κέρσορα μετατρέπεται σε συντεταγμένες SVG:

```
function getPointerPosition(event) {
    return d3.pointer(event, svg.node());
  }
```

Η συνάρτηση `startSelection()` ξεκινά την διαδικασία επιλογής:

```
function startSelection(event) {
    const ui = getUIState();
    if (ui.interactionMode !== "passed") return;
    if (event.target.tagName === "circle") return;

    dragSelection = true;
    movedSelection = false;
    selectionBegins = getPointerPosition(event);

    dragSelectionBox
      .attr("display", null)
      .attr("x", selectionBegins[0])
      .attr("y", selectionBegins[1])
      .attr("width", 0)
      .attr("height", 0);
  }
```

Αρχικά η συνάρτηση ελέγχει αν το Planning Mode είναι ενεργοποιημένο. Στην περίπτωση που δεν είναι, η διαδικασία επιλογής δεν ξεκινά. Επίσης, αποτρέπει την έναρξη του drag selection όταν ο χρήστης κάνει κλικ απευθείας πάνω σε έναν κόμβο μαθήματος.

Στη συνέχεια, ενεργοποιείται η κατάσταση drag selection, η αρχική θέση του δείκτη αποθηκεύεται και εμφανίζεται το ορθογώνιο επιλογής.

Όσο ο χρήστης μετακινεί το ποντίκι, το ορθογώνιο επιλογής ενημερώνεται δυναμικά μέσω της συνάρτησης `updateSelection()`:

```
function updateSelection(event) {
    if (!dragSelection) return;

    movedSelection = true;
    const [x, y] = getPointerPosition(event);

    const rectX = Math.min(selectionBegins[0], x);
    const rectY = Math.min(selectionBegins[1], y);
    const rectW = Math.abs(x - selectionBegins[0]);
    const rectH = Math.abs(y - selectionBegins[1]);

    dragSelectionBox
      .attr("x", rectX)
      .attr("y", rectY)
      .attr("width", rectW)
      .attr("height", rectH);
  }
```

Η συνάρτηση υπολογίζει το μέγεθος και την θέση του ορθογωνίου και με βάση την αρχική και την τρέχουσα θέση του κέρσορα.

Το drag selection ολοκληρώνεται με τη συνάρτηση `completeSelection()`:

```
function completeSelection(event) {
    if (!dragSelection) return;

    const ui = getUIState();
    const [x, y] = getPointerPosition(event);

    const rectX = Math.min(selectionBegins[0], x);
    const rectY = Math.min(selectionBegins[1], y);
    const rectW = Math.abs(x - selectionBegins[0]);
    const rectH = Math.abs(y - selectionBegins[1]);

    dragSelectionBox.attr("display", "none");

    if (movedSelection && rectW > 4 && rectH > 4) {
      nodes.forEach(d => {
        if (!passesAllFilters(d, ui)) return;

        if (
          d.x >= rectX &&
          d.x <= rectX + rectW &&
          d.y >= rectY &&
          d.y <= rectY + rectH
        ) {
          passedCourses.add(d.id);
        }
```

```
      });

      saveFocus();
      giveTagsPassedCourses();
      applyVisuals();
    }

    dragSelection = false;
    selectionBegins = null;
    movedSelection = false;
  }
```

Η συνάρτηση κρύβει το ορθογώνιο επιλογής και ελέγχει αν ο χρήστης έκανε drag selection. Έπειτα, ελέγχει όλους τους ορατούς κόμβους και εξετάζει ποιοι βρίσκονται μέσα στα όρια της επιλεγμένης περιοχής. Τα μαθήματα που βρίσκονται εντός, προστίθενται στο σύνολο `passedCourses`.

Μόλις ολοκληρωθεί το drag selection, το σύστημα αποθηκεύει την νέα κατάσταση, ενημερώνει τα tags μαθημάτων και ανανεώνει την οπτικοποίηση του γράφου. Τέλος, οι μεταβλητές μηδενίζονται, ώστε να είναι έτοιμες για το επόμενο drag selection.

5.11 Ενημέρωση οπτικής κατάστασης

Η οπτική εμφάνιση του γράφου ενημερώνεται δυναμικά μέσω της συνάρτησης `applyVisuals()`.

Η συνάρτηση είναι υπεύθυνη για τη διαχείριση της ορατότητας, των χρωμάτων και της επισήμανσης των κόμβων και των συνδέσεων, ανάλογα με το ενεργή λειτουργία αλληλεπίδρασης και τα επιλεγμένα φίλτρα.

Η ορατότητα των κόμβων και των labels ελέγχεται δυναμικά με βάση τα ενεργά φίλτρα:

```
node.style("display", d => (passesAllFilters(d, ui) ? null : "none"));
label.style("display", d => (passesAllFilters(d, ui) ? null : "none"));
```

Όταν η λειτουργία Focus Mode δεν είναι ενεργή, εφαρμόζονται οι κανόνες οπτικοποίησης του Planning Mode.

Όταν η λειτουργία του Focus Mode είναι ενεργή, τότε εφαρμόζονται οι κανόνες οπτικοποίησης του Focus Mode.

5.12 Βοηθητικά υπομνήματα και επεξηγηματικά tooltips

5.12.1 Legends

Τα βοηθητικά υπομνήματα ή αλλιώς legends, παρέχουν επεξηγήσεις σχετικά με τα χρώματα των κόμβων, τα οποία αλλάζουν ανάλογα με το ενεργό mode αλληλεπίδρασης της εφαρμογής.

Στο Focus Mode, τα legends δίνουν επεξηγήσεις για τα προαπαιτούμενα μαθήματα, το επιλεγμένο μάθημα και τα μαθήματα που ξεκλειδώνονται μέσω αυτού.

Στο Planning Mode εμφανίζονται πληροφορίες για τα ολοκληρωμένα, διαθέσιμα και μη διαθέσιμα μαθήματα. Ακόμα, εμφανίζεται η λίστα με τα tags των περασμένων μαθημάτων, καθώς και κουμπί επαναφοράς, σε περίπτωση που ο χρήστης θέλει να διαγράψει συνολικά όλα τα επιλεγμένα μαθήματα.

Η συνάρτηση `giveModeHelp()` είναι υπεύθυνη για αυτή την λειτουργία, η οποία ενημερώνει δυναμικά το περιεχόμενο των legends που βρίσκονται στο Πίνακα Ελέγχου.

```
function giveModeHelp() {
    const help = document.getElementById("mode-help-block");
    if (!help) return;

    const ui = getUIState();
```

Η δυναμική ενημέρωση των legends είναι πολύ, αφού μέσω αυτών, ο χρήστης μπορεί ευκολά να αντιληφθεί την σημασία των χρωμάτων και των στοιχείων του γράφου.

5.12.2 Tooltips

Τα tooltips λειτουργούν ως επεξηγηματική βοήθεια για τον χρήστη και έχουν τοποθετηθεί σε κάθε λειτουργία του Πίνακα Ελέγχου σε μορφή info icon ⓘ.

Τοποθετώντας τον κέρσορα πάνω από ένα από αυτά τα εικονίδια, εμφανίζεται ένα πλαίσιο με επεξηγηματικές πληροφορίες σε μορφή κειμένου.

Η λειτουργία των tooltips υλοποιείται μέσω του παρακάτω κώδικα:

```
const tooltipBox = document.getElementById("tooltip-box");

  document.querySelectorAll(".info-icon").forEach(icon => {

    icon.onmouseenter = () => {
      const html = icon.getAttribute("data-tooltip-html");
      if (!html || !tooltipBox) return;

      tooltipBox.innerHTML = html;
      tooltipBox.style.display = "block";
    };

    icon.onmousemove = (e) => {
      if (!tooltipBox) return;
      tooltipBox.style.left = (e.clientX + 15) + "px";
      tooltipBox.style.top = (e.clientY + 15) + "px";
    };

    icon.onmouseleave = () => {
```

```
      if (!tooltipBox) return;
      tooltipBox.style.display = "none";
    };
  });
```

Σκοπός αυτού του μηχανισμού είναι να βελτιώσει την χρηστικότητα της εφαρμογής καθώς παρέχει βοηθητικές πληροφορίες μόνο όταν ο χρήστης επιλέξει να τις δει.

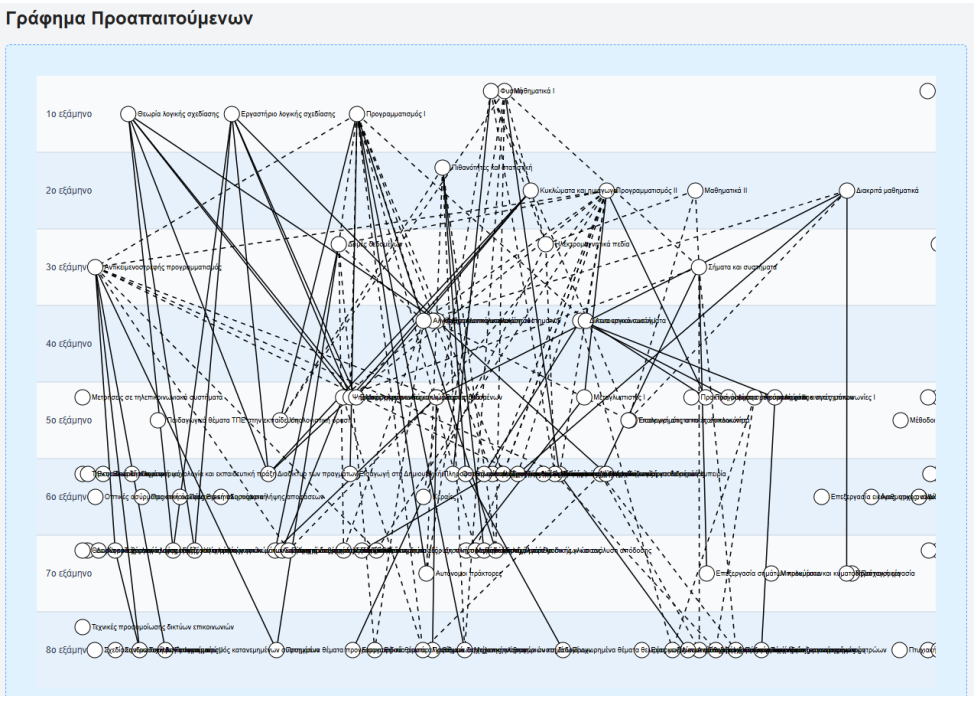
Κεφάλαιο 6

Εξέλιξη υλοποίησης και αρχικές ιδέες σχεδιασμού

Κατά την διάρκεια της υλοποίησης του συστήματος, σχεδιάστηκαν και δοκιμάστηκαν διαφορετικές ιδέες ως προς την οπτικοποίηση και τους μηχανισμούς αλληλεπίδρασης. Έχοντας ως κύριο στόχο την ευχρηστία και οπτική διαύγεια για την εφαρμογή μας, αρκετές από αυτές τις ιδέες απορρίφθηκαν, για να καταλήξει το σύστημα στην τελική του μορφή.

6.1 Αρχική δομή του γράφου

Αρχικά, η οπτικοποίηση του γράφου οργανωνόταν με βάση τα ακαδημαϊκά εξάμηνα και όχι τα ακαδημαϊκά έτη. Παρόλο που η συγκεκριμένη προσέγγιση είχε πιο ακριβή διαχωρισμό των μαθημάτων, ο γράφος γρήγορα έγινε οπτικά φορτωμένος λόγω του μεγάλου αριθμού εξαμήνων και συνδέσεων προαπαιτούμενων. Αυτό είχε ως αποτέλεσμα, η οπτικοποίηση να γίνει δύσκολη στην πλοήγηση και στην κατανόηση.

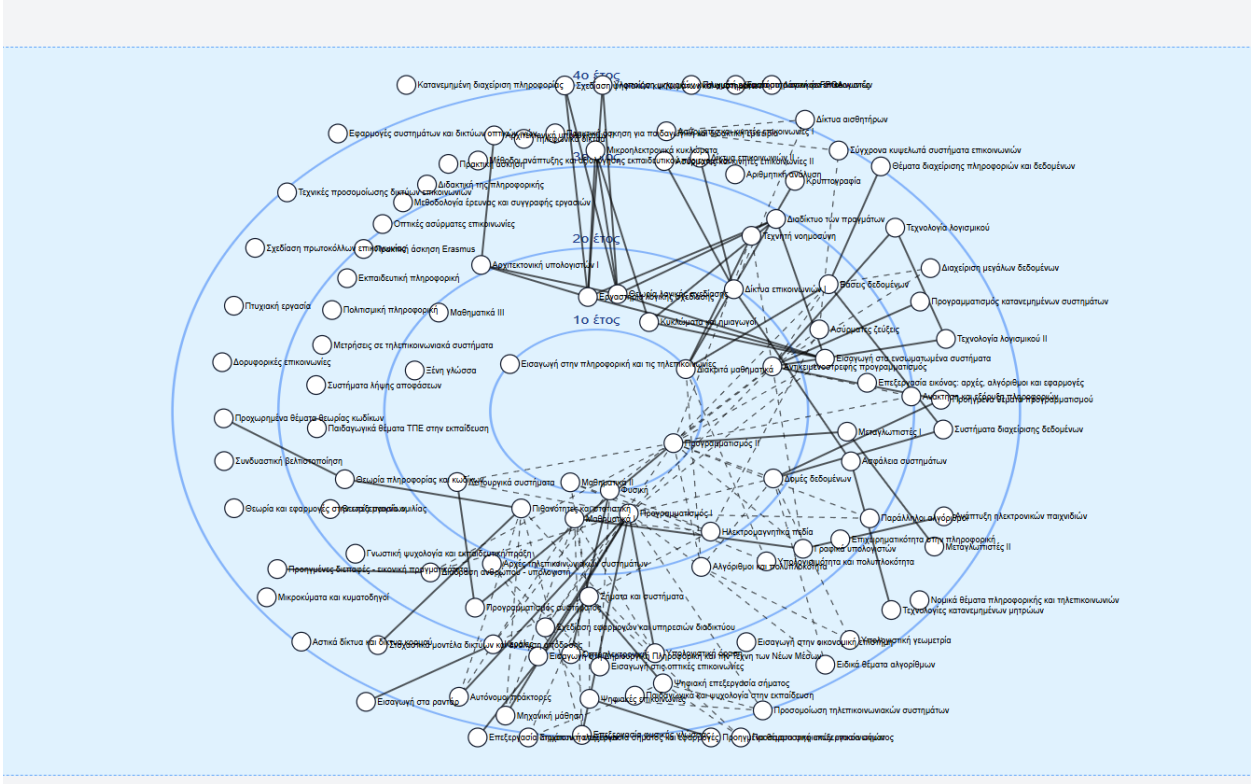


Οργάνωση γράφου ανά ακαδημαϊκά εξάμηνα

Η δομή του γραφήματος επανασχεδιάστηκε ώστε τα μαθήματα να οργανώνονται ανά ακαδημαϊκό έτος. Η αλλαγή αυτή βελτίωσε σημαντικά την αναγνωσιμότητα και δημιούργησε μια πιο καθαρή εικόνα.

6.2 Εναλλακτικές προσεγγίσεις οπτικοποίησης

Κατά την διάρκεια της υλοποίησης της εφαρμογής, εξετάστηκαν επίσης εναλλακτικές διατάξεις του γράφου. Μία από αυτές, βασίστηκε σε κυκλική δομή αποτελούμενη από τέσσερις ομόκεντρους κύκλους. Κάθε ακαδημαϊκό έτος αντιστοιχούσε σε διαφορετικό κύκλο, όπου στον εσωτερικό κύκλο βρισκόντουσαν τα μαθήματα του πρώτου έτους και στον εξωτερικό τα μαθήματα του τέταρτου.



Κυκλική δομή γράφου

Η αρχική ιδέα πίσω από την κυκλική διάταξη ήταν η καλύτερη αξιοποίηση του διαθέσιμου χώρου για την τοποθέτηση των κόμβων. Καθώς οι εξωτερικοί κύκλοι έχουν μεγαλύτερη περιφέρεια, προσφέρουν περισσότερο χώρο για την τοποθέτηση των μαθημάτων, τα οποία αυξάνονται σε αριθμό στα μεγαλύτερα ακαδημαϊκά έτη.

Η συγκεκριμένη προσέγγιση τελικά δεν διατηρήθηκε, καθώς όσο αυξανόταν ο αριθμός των κόμβων και των μεταξύ τους συνδέσεων, η απεικόνιση γινόταν περισσότερο περίπλοκη και δυσανάγνωστη. Οι συνδέσεις μεταξύ μαθημάτων διαφορετικών ετών διασταυρώνονταν σε μεγάλο βαθμό, με αποτέλεσμα να δυσκολεύεται η κατανόηση των σχέσεων προαπαιτούμενων.

Έτσι, επιστρέψαμε στην αρχική δομή του γράφου με την οριζόντια διάταξη, διατηρώντας όμως την οργάνωση του γράφου με βάση τα ακαδημαϊκά έτη. Η τελική διάταξη παρέχει μια πιο καθαρή και οργανωμένη εικόνα, καθώς τα μαθήματα παρουσιάζονται διαδοχικά από το πρώτο έως το τέταρτο ακαδημαϊκό έτος.

6.3 Λειτουργία μετακίνησης κόμβων

Μια ακόμα λειτουργία αλληλεπίδρασης με τον γράφο που δοκιμάστηκε, ήταν η δυνατότητα από τον χρήστη να μετακινεί ελευθέρα τους κόμβους με το ποντίκι, δηλαδή λειτουργία drag and drop.

Στόχος αυτής της λειτουργίας, ήταν να προσφέρει περισσότερη ευελιξία αλληλεπίδρασης και να επιτρέπει στους χρήστες να προσαρμόζουν την διάταξη του γράφου σύμφωνα με τις δικές τους προτιμήσεις. Ωστοσο, επειδή οι κόμβοι μπορούσαν να μετακινηθούν ελεύθερα, η δομή του γράφου συχνά αποδιοργανωνόταν και γινόταν δύσκολη στην ερμηνεία. Ένα γράφημα που συνεχώς μπορεί να αλλάξει, αφαιρεί από την οπτικοποίηση την σταθερότητα και την συνοχή της, δυσκολεύοντας τους χρήστες να διατηρήσουν μια σαφή αντίληψη της δομής των προαπαιτούμενων.

Για αυτούς τους λόγους, η λειτουργία μετακίνησης κόμβων αφαιρέθηκε από την τελική υλοποίηση. Στο τελικό σχέδιο που επιλέξαμε για την εφαρμογή, ο γράφος χρησιμοποιεί πλέον σταθερούς κανόνες τοποθέτησης των κόμβων που διατηρούν την δομή και την αναγνωσιμότητα της οπτικοποίησης.

6.4 Ετικέτες κόμβων

Αξίζει να αναφερθεί πως δοκιμάστηκαν διαφορετικές προσεγγίσεις για την εμφάνιση των ετικετών μαθημάτων μέσα στον γράφο. Στην αρχική οπτικοποίηση, εμφανιζόντουσαν οι πλήρεις τίτλοι των μαθημάτων δίπλα στους κόμβους του γράφου. Ωστόσο, πολλοί τίτλοι μαθημάτων ήταν ιδιαίτερα μεγάλοι, με αποτέλεσμα ο γράφος να γίνεται οπτικά φορτωμένος και δυσανάγνωστος.

Για αυτόν τον λόγο, επιλέξαμε στην τελική υλοποίηση να χρησιμοποιούνται οι κωδικοί των μαθημάτων, μειώνοντας σημαντικά την ποσότητα κειμένου μέσα στο γράφημα. Ευτυχώς οι

κωδικοί των μαθημάτων παραπέμπουν εύκολα στις ονομασίες τους (π.χ. «προ-1» για τον «Προγραμματισμό Ι») και έτσι ο χρήστης μπορεί εύκολα να αναγνωρίσει τα μαθήματα.

Οι πλήρεις τίτλοι των μαθημάτων παραμένουν διαθέσιμοι μέσω του panel πληροφοριών, στα οποία ο χρήστης έχει πρόσβαση πατώντας διπλό κλικ σε έναν κόμβο. Έτσι, η εφαρμογή διατηρεί σε ισορροπία την απλότητα και την πληρότητα πληροφορίας.

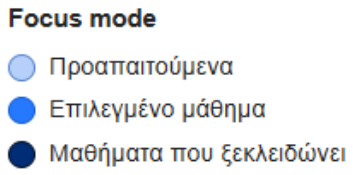
6.5 Χρωματικές παλέτες εφαρμογής

Οι χρωματικές παλέτες που χρησιμοποιήθηκαν στην εφαρμογή επιλέχθηκαν με μεγάλη προσοχή, με στόχο την καθαρότητα, την συνέπεια και την άμεση κατανόηση των οπτικών στοιχείων από τον χρήστη.

Όσον αφορά την γενική αισθητική της εφαρμογής, η χρωματική παλέτα που χρησιμοποιήθηκε είναι η ιδιά με αυτήν της επίσημης ιστοσελίδας του πανεπιστημίου και του τμήματος. Η επιλογή αυτή έγινε με στόχο την δημιουργία ενός πιο οικείου και οπτικά συνεπούς περιβάλλοντος για τους φοιτητές που χρησιμοποιούν την εφαρμογή.

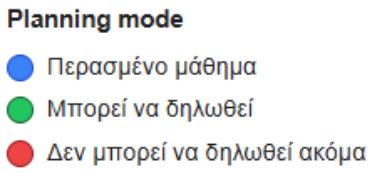
Για τις δυο βασικές λειτουργίες αλληλεπίδρασης (focus mode και planning mode) του συστήματος χρησιμοποιήθηκαν διαφορετικές χρωματικές παλέτες.

Για το Focus Mode χρησιμοποιήθηκαν διαφορετικές αποχρώσεις του μπλε, με αλλαγές στην ένταση και στην φωτεινότητα του χρώματος. Η προσέγγιση αυτή δημιουργεί οπτική συνοχή μεταξύ του επιλεγμένου μαθήματος, των προαπαιτούμενων μαθημάτων και των μαθημάτων που ξεκλειδώνονται μέσω αυτού. Παράλληλα, η χρήση αποχρώσεων του ίδιου χρώματος υποδηλώνει συσχετισμό μεταξύ των συγκεκριμένων κόμβων και βοηθά τον χρήστη να αντιλαμβάνεται ότι όλοι οι επισημασμένοι κόμβοι ανήκουν στην ίδια δομή προαπαιτούμενων.



Χρωματική παλέτα Focus Mode

Για το Planning Mode χρησιμοποιήθηκαν τρία διαφορετικά χρώματα για την σαφή αναπαράσταση της κατάστασης του κάθε μαθήματος. Μπλε για τα ολοκληρωμένα μαθήματα, πράσινο για τα διαθέσιμα μαθήματα και κόκκινο για τα μη διαθέσιμα μαθήματα.



Χρωματική παλέτα Planning Mode

Οι συγκεκριμένες επιλογές βασίστηκαν σε οπτικές συσχετίσεις από το γενικό κοινό και στην εξοικείωση των χρηστών με τη σημασία των χρωμάτων από την καθημερινή ζωή. Το μπλε χρησιμοποιήθηκε ως ουδέτερο χρώμα για τα ολοκληρωμένα μαθήματα, το πράσινο συνδέθηκε με θετική / επιτρεπτή κατάσταση, ενώ το κόκκινο χρησιμοποιήθηκε για να δηλώνει περιορισμό / μη διαθέσιμη κατάσταση μαθήματος.

Κεφάλαιο 7

Συμπεράσματα και μελλοντικές βελτιώσεις

7.1 Συμπεράσματα

Κλείνοντας αυτή την εργασία, θα θέλαμε να αναφερθούμε σε κάποια συμπεράσματα, τα οποία καταλήξαμε υλοποιώντας τόσο το πρακτικό κομμάτι της εργασίας όσο και το θεωρητικό. Η εφαρμογή που αναπτύχθηκε παρέχει έναν πιο οργανωμένο και φιλικό προς τον χρήστη τρόπο εξερεύνησης του οδηγού σπουδών. Μέσω της εφαρμογής, οι φοιτητές θα μπορέσουν να κατανοήσουν καλύτερα τις εξαρτήσεις των μαθημάτων και να σχεδιάζουν πιο αποτελεσματικά την ακαδημαϊκή τους πορεία.

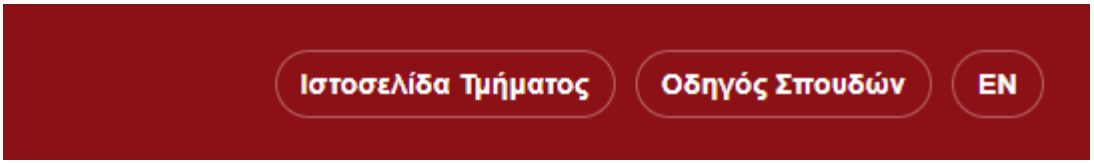
Παράλληλα, μέσα από την ανάπτυξη της εφαρμογής έγινε φανερό ότι οι τεχνικές οπτικοποίησης γράφων και οι σύγχρονες web τεχνολογίες μπορούν να αξιοποιηθούν αποτελεσματικά για την δημιουργία διαδραστικών εκπαιδευτικών συστημάτων. Τέτοιου είδους εφαρμογές οπτικοποίησης μπορούν και πρέπει να αναπτυχθούν και για άλλα Πανεπιστημιακά Ιδρύματα, για την καλύτερη καθοδήγηση των φοιτητών κατά τη διάρκεια της ακαδημαϊκής τους πορείας.

7.2 Μελλοντικές βελτιώσεις

Παρόλο που η εφαρμογή ήδη παρέχει ένα λειτουργικό και διαδραστικό σύστημα οπτικοποίησης προαπαιτούμενων, υπάρχουν αρκετές δυνατότητες μελλοντικής βελτίωσης και επέκτασης.

Μια πιθανή επέκταση θα μπορούσε να είναι η προσθήκη προσωποποιημένων προφίλ φοιτητών. Ο κάθε φοιτητής θα μπορεί να συνδέεται στην εφαρμογή με το ακαδημαϊκό του email και κωδικό πρόσβασης, αποθηκεύοντας την ακαδημαϊκή πρόοδο μέσω συστήματος βάσης δεδομένων. Ακόμα, η πρόσβαση στα μαθήματα που έχει επιτυχώς ολοκληρώσει ο φοιτητής, θα διευκολύνει την χρηστικότητα του Planning Mode, καθώς τα περασμένα μαθήματα θα εμφανίζονται αυτομάτως στην εφαρμογή, χωρίς να χρειάζεται να τα επιλέξει ο ίδιος ο φοιτητής.

Επιπλέον, θα μπορούσε να προστεθεί η δυνατότητα αλλαγής γλώσσας στην διεπαφή της εφαρμογής, με υποστήριξη αγγλικής έκδοσης. Η δυνατότητα αυτή θα επιτρέπει την χρήση του συστήματος και από αγγλόφωνους χρήστες.



Κουμπί EN για αλλαγή στα Αγγλικά

Τέλος, θα μπορούσε να πραγματοποιηθεί μια αξιολόγηση της εφαρμογής από ομάδα φοιτητών του τμήματος. Με αυτόν τον τρόπο θα μπορούσε να εξεταστεί κατά πόσο η οπτικοποίηση των μαθημάτων και των προαπαιτούμενων διευκολύνει την κατανόηση του προγράμματος σπουδών. Παράλληλα, τα σχόλια των φοιτητών θα μπορούσαν να βοηθήσουν στον εντοπισμό σημείων που χρειάζονται βελτίωση.

Παράρτημα Α

Τοπική εκτέλεση της εφαρμογής

Η εφαρμογή αναπτύχθηκε και δοκιμάστηκε σε τοπικό περιβάλλον ανάπτυξης χρησιμοποιώντας τον Apache web server μέσω του XAMPP και PHP 8.0.30

Για την εκτέλεση της εφαρμογής τοπικά απαιτούνται τα εξής βήματα:

1. Εγκατάσταση του XAMPP.
2. Ενεργοποίηση του Apache server μέσω του XAMPP Control Panel.
3. Τοποθέτηση των αρχείων της εφαρμογής στον φάκελο htdocs.
4. Πρόσβαση στην εφαρμογή μέσω web browser.

Η εφαρμογή μπορεί να εκτελεστεί μέσω της διεύθυνσης:

http://localhost/thesis/index.php

Η SQLite βάση δεδομένων περιλαμβάνεται τοπικά στην εφαρμογή και δεν απαιτείται ξεχωριστή εγκατάσταση κάποιου database server.

Βιβλιογραφία

[1] «SQLite», Wikipedia, https://el.wikipedia.org/wiki/SQLite

[2] «SVG files», Adobe, https://www.adobe.com/gr_el/creativecloud/file-types/image/vector/svg-file.html

[3] «Force Simulations», d3js, https://d3js.org/d3-force/simulation

[4] Οπτικοποίηση Δεδομένων με D3.js. https://observablehq.com/@d3/gallery

[5] «Introducing JSON», JSON.org, https://www.json.org/json-en.html

[6] «PHP», Wikipedia, https://en.wikipedia.org/wiki/PHP

[7] «HTML», Wikipedia, https://en.wikipedia.org/wiki/HTML

[8] «CSS», Wikipedia, https://el.wikipedia.org/wiki/CSS

[9] «JavaScript», Wikipedia, https://en.wikipedia.org/wiki/JavaScript

[10] «PHP Data Objects», php.net, https://www.php.net/manual/en/book.pdo.php